

CSE-214 Exam Study Notes

Design Patterns · Git · Clean Code · Refactoring · Load Testing

Detailed edition — every concept is explained in depth: the motivating problem, the reasoning that leads to the solution, the structure, code, trade-offs, and how it compares to related concepts. Practice questions modeled on your reference quiz follow every topic.

1. Creational Design Patterns

Creational design patterns are concerned with the mechanics of object creation. Instead of instantiating objects directly with the `new` keyword scattered everywhere in client code, creational patterns push the decision of "what to create" and "how to create it" into a dedicated place. This keeps a system flexible (new kinds of objects can be introduced without rewriting client code), reusable (the creation logic is written once), and decoupled (client code depends on abstractions/interfaces, not concrete classes).

Why does this matter? If client code is littered with statements like `new ConcreteProductA()`, then every time you need a new variant of that product, you must find and edit every place that used `new`. That violates the Open/Closed Principle ("open for extension, closed for modification"). Creational patterns solve this by introducing one seam — a factory, a builder, a single instance accessor — through which all object creation flows.

1.1 Factory Method

Intent: Define an interface for creating an object, but let subclasses decide which class to instantiate. Factory Method lets a class defer instantiation to subclasses.

The problem in detail

Imagine you are building a logistics management application. The first version only needs to handle transportation by trucks, so almost all of your business logic (route planning, cost calculation, delivery confirmation) lives inside a `Truck` class or is tightly coupled to it. The app becomes popular, and customers start asking for sea transportation support. Now you face a dilemma: most of your code assumes a `Truck`. Adding a `Ship` class means hunting down every place that says `new Truck()` and wrapping it in a conditional (`if (type == SEA) new Ship(); else new Truck();`). Every time a new transport type (train, drone) is added, you must revisit and edit that same conditional again, and the code becomes riddled with `if/else` or `switch` statements that grow forever and are easy to break.

The underlying problem is that the code which USES the product (e.g., "plan a delivery route") is mixed together with the code that CREATES the product ("decide whether it's a `Truck` or a `Ship`"). These are two different responsibilities and they change for different reasons — a new transport type is a creation-time change, while a new business rule is a usage-time change. Mixing them violates the Single Responsibility Principle.

The solution

Factory Method suggests replacing direct object-construction calls with calls to a special factory method. Creating an object directly inside the method that needs it is not itself a problem — the problem is that the method needing the object typically also contains a large body of logic, and that logic should not have to change every time a new product type is introduced. The Factory Method pattern extracts the creation step into its own method, and lets subclasses of the creator override that one method to decide the concrete product class.

- **Product** — the interface common to all objects the factory can produce (e.g., `Transport`, `Shape`).
- **Concrete Products** — different implementations of the Product interface (`Truck`, `Ship` / `Circle`, `Rectangle`, `Square`).
- **Creator** — declares the factory method that returns Product objects; it may also contain core business logic that operates on Products returned by the factory method, but it does not know which concrete product it will get.
- **Concrete Creator** — overrides the base factory method to return a different type of product (`RoadLogistics` returns a `Truck`; `SeaLogistics` returns a `Ship`).

Worked example: Shape drawing

```
public interface Shape {
    void draw();
}
public class Rectangle implements Shape {
    @Override
    public void draw() {
        System.out.println("Inside Rectangle::draw() method.");
    }
}
// Circle and Square implement Shape the same way.

public class ShapeFactory {
    public Shape getShape(String type) {
        if ("CIRCLE".equalsIgnoreCase(type)) return new Circle();
        if ("RECTANGLE".equalsIgnoreCase(type)) return new Rectangle();
        if ("SQUARE".equalsIgnoreCase(type)) return new Square();
        return null;
    }
}

// Client code only depends on Shape and ShapeFactory:
Shape shape = new ShapeFactory().getShape("CIRCLE");
shape.draw();
```

Notice the client never writes `new Circle()` directly. If a `Triangle` is added later, only `ShapeFactory` needs to change; every piece of client code that already calls `factory.getShape(...)` keeps working untouched.

Worked example: Kiosk payment

A Kiosk needs to accept payment by Cash, Coupon, or CreditCard. All three implement a `PaymentProcessor` interface with `processPayment()`. A `GetPayment` factory exposes `getProcessor(type): PaymentProcessor`, hiding which concrete class is used. Adding a fourth payment method (e.g., `MobileWallet`) only means writing a new class and adding one branch in the factory — the Kiosk's checkout logic never changes.

Extending a framework — the library-author's use case

Factory Method is also the standard way to let users of your library or framework extend its internal components. Suppose you are using an open-source UI framework whose buttons are always square, but your app requires round buttons. You cannot edit the framework's source, so you: (1) create a `RoundButton` subclass of the framework's `Button` class; (2) create a subclass of the framework's main UI class (say, `UIWithRoundButtons` extends `UIFramework`) and override its `createButton()` factory method so that it returns a `RoundButton` instead of a plain `Button`; (3) use `UIWithRoundButtons` instead of `UIFramework` everywhere. The framework's own internal code, which calls `createButton()` and only knows about the `Button` interface, keeps working — it just receives round buttons now instead of square ones.

Benefits and trade-offs

- Code to an interface, not an implementation — client code manipulates `Shape/PaymentProcessor/Transport`, never the concrete class.
- Removes the responsibility of instantiating concrete implementation classes from the client code, making the code more robust, less coupled, and easier to extend.
- Provides a hook for subclasses to provide an extended version of an object, since the creator class doesn't know ahead of time the actual concrete class it will create.
- Trade-off: the code can become more complicated because you need to introduce a lot of new subclasses to implement the pattern — it is only worth it when you actually anticipate needing new product variants.

1.2 Abstract Factory

Intent: Provide an interface for creating families of related or dependent objects without specifying their concrete classes.

The problem in detail

Imagine you are building a furniture-shop simulator. Your code deals with a family of related products: Chair, Sofa, and CoffeeTable. Each product also comes in several style variants: Modern, Victorian, ArtDeco. You need a way to create individual furniture objects so that they always match — that is, so that a Modern chair is never accidentally paired with a Victorian sofa in the same room, because customers get quite mad when they receive non-matching furniture. Additionally, when a vendor updates its catalog and adds a new style, you don't want to have to hunt through and edit the core application code.

A naive approach might use a giant `if/else` on the style everywhere a furniture piece is created, but that duplicates the same conditional in every place furniture is instantiated, and adding a new style means finding and editing every one of those places (this is precisely the same underlying problem Factory Method solves for a single product — Abstract Factory generalizes it to a whole family of related products).

The solution, step by step

- Step 1 — declare interfaces for each distinct product in the family: Chair, Sofa, CoffeeTable. All variants of a chair (ModernChair, VictorianChair, ...) implement the Chair interface, and likewise for the other products.
- Step 2 — declare the Abstract Factory: an interface with a creation method for every product in the family, e.g. createChair(), createSofa(), createCoffeeTable().
- Step 3 — for every variant of the product family, create a separate concrete factory class based on the AbstractFactory interface. A factory is a class that returns products of one particular style — e.g., ModernFactory only ever creates ModernChair, ModernSofa, and ModernCoffeeTable objects, guaranteeing that everything it produces matches.
- Step 4 — client code works with factories and products only through their abstract interfaces (AbstractFactory, Chair, Sofa, CoffeeTable). This lets the same client function work with any factory/product family passed into it.

Worked example: Car parts factory

```
interface Engine { }
interface Transmission { }

class CivicEngine implements Engine { }
class CivicTransmission implements Transmission { }
class AccordEngine implements Engine { }
class AccordTransmission implements Transmission { }

interface HondaFactory {
    Engine createEngine();
    Transmission createTransmission();
}
class CivicFactory implements HondaFactory {
    public Engine createEngine() { return new CivicEngine(); }
    public Transmission createTransmission() { return new CivicTransmission(); }
}
class AccordFactory implements HondaFactory {
    public Engine createEngine() { return new AccordEngine(); }
    public Transmission createTransmission() { return new AccordTransmission(); }
}

// Client only knows HondaFactory, Engine, Transmission:
```

```
HondaFactory factory = new CivicFactory();
Engine e = factory.createEngine();           // always matches
Transmission t = factory.createTransmission(); // the same model
```

Another common use case: cross-platform UI

When an application launches, it checks the current operating system and creates one concrete factory that matches it (e.g., WindowsFactory, MacFactory, LinuxFactory). All later UI-element creation goes through that one factory object, so the rest of the code never needs to know or care which OS it's running on — it only depends on abstract interfaces like Button and Checkbox. This also lets you add support for a brand-new OS, or a brand-new UI toolkit, later, without touching any of the code that already uses the abstract interfaces.

Consequences

- Isolates concrete classes from the client.
- Makes exchanging entire product families easy — swap one factory instance for another.
- Promotes consistency among products, since a single factory guarantees a matching family.
- It is often called a "factory of factories" and can be easily extended to accommodate more products, and it avoids the conditional/branching logic that plain Factory pattern implementations can accumulate.
- Trade-off: supporting a brand-new KIND of product (not just a new style of existing products) is difficult, because it requires changing the AbstractFactory interface itself and therefore every concrete factory that implements it.

Factory Method vs. Abstract Factory

Factory Method creates ONE product via a single overridable method and relies on subclassing (inheritance) to vary the product. Abstract Factory creates an entire FAMILY of related products via a factory object and relies on object composition — you plug in a different concrete factory OBJECT rather than subclassing. In fact, Abstract Factory is very often implemented using several Factory Methods internally: each creation method on the AbstractFactory interface (createChair, createSofa...) is itself a Factory Method.

1.3 Singleton

Intent: Ensure a class has only one instance, and provide a single, global point of access to that instance — typically with lazy ("just-in-time") initialization, meaning the instance is only created the first time it is actually requested, not automatically when the program starts.

The problem in detail

Some objects genuinely need to be unique across the whole application: a single configuration object, a single database-connection manager, a single logging facility, a single thread pool. If any code can call `new DbConnection()` freely, you could end up with dozens of independent connection objects, wasting resources and potentially producing inconsistent behavior (e.g., two different connection objects with two different transaction states). A plain global variable does not fully solve this either, because nothing stops another part of the code from creating a second object and assigning it over the global variable, and global variables are also visible and mutable from everywhere, which is bad for encapsulation.

The solution

Make the constructor private so that no code outside the class itself can call `new`. Provide a static field to hold the one instance, and a static accessor method (traditionally called `getInstance()`) that creates the object the first time it's called and returns the same cached object on every subsequent call.

```
public class DbConnection {
    private static DbConnection instance = null;
    private SqlConnection connection;
```

```

private DbConnection() { // constructor is PRIVATE
    connection = connectToDatabase(dbUser, dbPassword, dbName);
}

public static DbConnection getDbConnection() { // the single global access point
    if (instance == null) {
        instance = new DbConnection(); // created lazily, on first use
    }
    return instance;
}
}

// Usage anywhere in the app:
DbConnection connection = DbConnection.getConnection();

```

Because the constructor is private, the ONLY way to obtain a `DbConnection` object anywhere in the codebase is through `getDbConnection()`, and that method always hands back the exact same object after the first call.

Applicability and caveats

- Use it when a class must have exactly one instance available to all clients (e.g., a single access point to some shared resource, like a database or a file).
- Unlike a global variable, Singleton guarantees uniqueness — nothing except the Singleton class itself can replace the cached instance.
- Common exam caveat: in multi-threaded programs, the naive `if (instance == null) instance = new ...` check is NOT thread-safe — two threads could both see `instance == null` at the same time and both create separate objects. Real implementations add synchronization or use eager initialization to avoid this race condition.
- Overuse caution: Singleton is sometimes criticized as a disguised global variable that can make unit testing harder (since it's difficult to substitute the singleton with a mock/fake in tests) and can hide dependencies between classes. Use it deliberately, not as a default way to avoid passing objects around.

Consequences

- Controlled access to the sole instance.
- Reduced namespace pollution compared to using global variables everywhere.
- Permits a variable number of instances if the design is later relaxed (e.g., a pool of at most N instances) — the pattern can be adapted, it isn't strictly limited to exactly one forever.

1.4 Builder

Intent: Separate the construction of a complex object from its representation, so that the same step-by-step construction process can create different representations of the object.

The problem in detail

Some objects require many steps to construct, and the exact combination of steps (and their parameters) varies from one instance to another. A naive approach is a single class constructor with a huge number of parameters (a "telescoping constructor" with 8, 10, or 12 parameters, many of them optional), which is hard to read at the call site ("what does the 6th `true` mean?") and error-prone (easy to swap two parameters of the same type by accident). Alternatively, you might try creating many subclasses for every possible combination of options, which explodes combinatorially just like the problems Factory patterns solve.

The solution

Extract the object-construction code out of the product's own class and move it into separate objects called builders. Each builder implements a common construction interface with methods for each configurable part of

the product (e.g., `setCPU()`, `setRAM()`, `setStorage()`, `setGPU()`). The client calls only the setter methods it actually needs, in whatever order makes sense, and then calls a final `build()` (or `getResult()`) method to retrieve the finished object. This lets you build very different representations of a product using the exact same sequence of construction steps — e.g., a "budget PC builder" and a "gaming PC builder" both follow the same overall assembly process but choose different concrete parts.

Worked example: Custom PC assembly

To create a computer, different parts are assembled depending on the order received from the customer — e.g., one customer wants a 500GB hard disk with an Intel processor, while another wants a 250GB hard disk with an AMD processor. A `ComputerBuilder` exposes methods like `setHardDisk(size)`, `setProcessor(type)`, `setRAM(size)`; calling them in sequence and finally calling `build()` produces the finished `Computer` object, with the same overall assembly process each time.

An optional Director

In the full GoF version of Builder, an optional `Director` class encapsulates common, reusable build recipes (e.g., `buildGamingPC(builder)` calls a fixed sequence of the builder's setter methods). This lets you reuse the exact same construction sequence to build different product representations by handing the `Director` a different concrete builder each time.

Consequences

- Lets you vary a product's internal representation without changing the client's construction code.
- Isolates the code for construction from the code that represents the finished business object — construction logic doesn't leak into the product class itself.
- Gives you finer control over the construction process than a single, all-at-once constructor — you can build a product step by step, skip steps that aren't needed, and even build the object in stages over time.

1.5 UML notation used across all patterns (quick reference)

These four relationship types recur constantly in design-pattern diagrams and are frequently tested directly, so make sure you can both draw them and recognize them:

- Generalization — an "IS-A" relationship: inheritance between classes, or a class implementing an interface. Drawn as a solid line with a hollow (unfilled) triangular arrowhead pointing at the parent/interface.
- Dependency — "uses temporarily": one class's method refers to another class only as, e.g., a parameter type or a local variable, without holding a long-lived reference to it. Drawn as a dashed line with an open arrowhead.
- Composition — "is entirely made of": a strong whole/part relationship where the parts cannot exist independently of the whole and are destroyed along with it (e.g., a `Book` is composed of `Pages` — delete the book, the pages are gone too). Drawn with a filled (black) diamond at the whole's end.
- Aggregation — "has a"/"is part of", a weaker whole/part relationship where the part CAN exist independently of, and be shared by, more than one whole (e.g., a `Department` has `Professors`, but a `Professor` still exists if the department is dissolved). Drawn with a hollow (white) diamond at the whole's end.

Practice Questions — Creational Patterns

A. True/False — write T or F.

Statement	T / F
-----------	-------

1. The Factory Method pattern lets a class defer instantiation of an object to its subclasses.	
2. Abstract Factory deals with exactly one factory, while Factory Method and Simple Factory can deal with multiple factories.	
3. A Singleton class exposes a public constructor so any client can create a new instance whenever needed.	
4. The Builder pattern is useful when the same construction process must produce different representations of a complex object.	
5. In the Abstract Factory pattern, every concrete factory must produce a full, consistent family of related products.	
6. Factory Method relies primarily on inheritance (overriding a method in a subclass), while Abstract Factory relies primarily on composition (swapping in a different factory object).	

B. Define the following terminologies in brief:

Term	Definition (write yours)
Factory Method	
Abstract Factory	
Singleton	
Builder	
Lazy Initialization	

C. Scenario identification (identify the pattern, state the reason in no more than 10 words):

1. A car manufacturer has separate assembly lines that each build a complete Sedan (matching engine, seats, and body) or a complete SUV (matching engine, seats, and body) — never a mismatched combination.
2. A logging utility must guarantee that the entire application writes to exactly one shared log file handle.
3. A meal-ordering app assembles a Combo Meal step-by-step (choose burger, choose side, choose drink) and the exact same steps can produce a Kids Combo or a Family Combo.

D. Short answer:

1. Explain, with an example, why the Abstract Factory pattern is sometimes called "a factory of factories."
2. Give one real-world class from a framework you know (or invent one) that should use the Singleton pattern, and justify your choice.
3. Explain why a naive Singleton implementation can fail in a multi-threaded program, and describe one way to fix it.

2. Structural Design Patterns

Structural patterns explain how to assemble objects and classes into larger structures, while keeping those structures flexible and efficient. Where creational patterns focus on HOW objects are created, structural patterns focus on HOW objects are composed together to form bigger pieces of functionality, usually by favoring object composition over class inheritance.

2.1 Decorator

Intent: Attach new behaviors to objects dynamically, at runtime, by placing them inside special wrapper objects that contain the added behavior — without altering the original object's class or affecting other objects of the same class.

The problem in detail (Starbuzz Coffee)

A coffee shop starts with a simple menu of beverages (HouseBlend, DarkRoast, Decaf, Espresso), each with its own `cost()` method. Then the shop needs to support condiments — steamed milk, soy, mocha, whipped cream — that customers can add or not add, and even add MULTIPLE TIMES (e.g., a double mocha). The first instinct is subclassing: create `DarkRoastWithMocha`, `DarkRoastWithMochaAndWhip`, and so on for every combination. This causes a subclass explosion — the number of subclasses grows combinatorially with the number of condiments, and adding a single new condiment means creating a whole new set of combination subclasses.

A second instinct is to move condiment logic into the abstract `Beverage` superclass using boolean flags (`hasMocha`, `hasWhip`, `hasSoy`...) and computing the total `cost()` in one place. This has three serious problems: (1) adding a new condiment means editing the shared superclass and every subclass that inherits it, breaking the Open/Closed Principle; (2) some beverages (like iced tea) might not support certain condiments at all, but they would still inherit the flags and the logic for them, which is a bad design smell; (3) it still cannot represent "the same condiment applied twice" (double mocha) since a boolean is either on or off, not a count.

The solution: decoration

Instead of trying to represent every combination as its own class, Decorator starts with a plain `Beverage` object and wraps it with one `Condiment` decorator object per condiment the customer ordered. Each decorator: (1) implements the SAME interface/abstract type as the object it wraps (so it can be used anywhere a `Beverage` is expected, and can even wrap ANOTHER decorator); (2) HAS-A reference to the wrapped `Beverage` object; (3) adds its own behavior (its own share of the cost, its own description text) and then delegates the rest of the call to the wrapped object.

```
abstract class Beverage {
    String description = "Unknown Beverage";
    public String getDescription() { return description; }
    public abstract double cost();
}

abstract class CondimentDecorator extends Beverage {
    protected Beverage beverage;    // the wrapped object
}

class DarkRoast extends Beverage {
    public DarkRoast() { description = "Dark Roast Coffee"; }
    public double cost() { return 0.99; }
}

class Mocha extends CondimentDecorator {
    public Mocha(Beverage beverage) { this.beverage = beverage; }
    public String getDescription() { return beverage.getDescription() + ", Mocha"; }
    public double cost() { return beverage.cost() + 0.20; }    // adds its own cost, then delegates
}
```

```

}

class Whip extends CondimentDecorator {
    public Whip(Beverage beverage) { this.beverage = beverage; }
    public String getDescription() { return beverage.getDescription() + ", Whip"; }
    public double cost() { return beverage.cost() + 0.10; }
}

// A DarkRoast with a DOUBLE Mocha and Whip:
Beverage order = new DarkRoast();
order = new Mocha(order);    // wrap once
order = new Mocha(order);    // wrap again -> represents a double mocha
order = new Whip(order);
System.out.println(order.getDescription() + " $" + order.cost());

```

Notice that wrapping Mocha twice naturally represents a double mocha, and no new class was needed — this solves the exact limitation the boolean-flag approach had. The client code that calls `cost()` and `getDescription()` never needs to know how many layers of wrapping exist underneath; each layer just does its bit and passes the call down the chain.

Consequences

- Behavior can be added or removed at runtime, and combined freely, avoiding the fixed, static subclass explosion of pure inheritance.
- Follows the Open/Closed Principle — new condiments/behaviors are added as new decorator classes, without touching existing Beverage or decorator classes.
- Trade-off: a long chain of small decorator objects can be harder to debug/understand than a single class, and the order in which decorators are applied can sometimes matter (be careful with decorators that aren't truly independent of each other).

2.2 Composite

Intent: Compose objects into tree structures to represent part-whole hierarchies, then let client code work with individual objects and compositions of objects uniformly — through the SAME interface.

The problem in detail

Consider a restaurant's menu system: a Menu can contain individual MenuItem's, but it can ALSO contain SUBMENUS (e.g., a "Desserts" submenu inside the main dinner menu), and those submenus can themselves contain further submenus. You need to be able to iterate over just one submenu, or over the entire menu hierarchy, without writing completely different code for "a single item" versus "a menu that contains other menus." If client code has to check "is this a MenuItem or a Menu?" everywhere and branch accordingly, the code becomes littered with type checks and is fragile to new levels of nesting.

The solution

Define one common component interface (e.g., MenuComponent) declaring the operations that make sense for both leaves and containers (e.g., print(), getPrice()). A Leaf (MenuItem) implements this interface directly with its own concrete data. A Composite (Menu) ALSO implements the SAME interface, but internally it holds a list of child MenuComponent objects (which may themselves be leaves or further composites) and implements each operation by looping over its children and calling the same operation on each of them recursively.

```

interface MenuComponent {
    void print();
    double getPrice();
}

class MenuItem implements MenuComponent {                // LEAF
    private String name; private double price;

```

```

    public MenuItem(String name, double price) { this.name = name; this.price = price; }
    public void print() { System.out.println(name + " - $" + price); }
    public double getPrice() { return price; }
}

class Menu implements MenuComponent { // COMPOSITE
    private List<MenuComponent> children = new ArrayList<>();
    private String name;
    public Menu(String name) { this.name = name; }
    public void add(MenuComponent c) { children.add(c); }
    public void print() {
        System.out.println("--- " + name + " ---");
        for (MenuComponent c : children) c.print(); // works whether c is a MenuItem OR a
        nested Menu
    }
    public double getPrice() {
        double total = 0;
        for (MenuComponent c : children) total += c.getPrice();
        return total;
    }
}

```

Client code that calls `print()` or `getPrice()` on a MenuComponent works correctly whether that component is a single MenuItem or an entire nested Menu — it does not need to know which one it has. This is the essence of Composite: "treat individual objects and compositions of objects uniformly."

Consequences

- Simplifies client code, since it can treat complex tree structures uniformly.
- Makes it easy to add new kinds of components (as long as they implement the common interface) without changing existing code.
- Trade-off: it can be harder to restrict what kinds of components are allowed inside a particular composite (the type system alone won't stop you from, say, adding a Menu inside a MenuItem if MenuItem is designed as a container too, so this must be enforced by design/documentation).

2.3 Adapter

Intent: Convert the interface of a class into another interface that clients expect. Adapter lets classes work together that otherwise could not, because of incompatible interfaces.

The problem in detail (Duck Simulator + Turkey)

Suppose your existing Duck Simulator code only knows about a Duck interface with methods quack() and fly(). A third-party vendor gives you a Turkey class with completely different method names — gobble() instead of quack(), and its fly() only manages a short burst of flight instead of the sustained flight expected from a Duck. You want to reuse the Turkey class inside code that only understands Duck, but you cannot (and should not) modify either the existing Duck-based code or the vendor's Turkey class.

The solution

Create a TurkeyAdapter class that IMPLEMENTS the Duck interface (the interface the client expects) and internally HOLDS a reference to a Turkey object (the interface that actually exists). Every method the client calls on the adapter (quack(), fly()) is translated into the equivalent call(s) on the wrapped Turkey object (gobble(), calling turkey.fly() multiple times to simulate a duck's longer flight).

```

interface Duck { void quack(); void fly(); }

class Turkey { // vendor's incompatible class
    public void gobble() { System.out.println("Gobble gobble"); }
    public void fly()    { System.out.println("I'm flying a short distance"); }
}

```

```

}

class TurkeyAdapter implements Duck {
    Turkey turkey;
    public TurkeyAdapter(Turkey turkey) { this.turkey = turkey; }
    public void quack() { turkey.gobble(); }           // translate the call
    public void fly() {
        for (int i = 0; i < 5; i++) turkey.fly();      // 5 short flights ~ 1 duck flight
    }
}

// Existing code that only knows Duck can now use a Turkey:
Duck turkeyAsDuck = new TurkeyAdapter(new Turkey());
turkeyAsDuck.quack();    // -> "Gobble gobble"

```

Object Adapter vs. Class Adapter

Aspect	Object Adapter	Class Adapter
Mechanism	Composition — the adapter implements the target interface and HOLDS a reference to (wraps) the adaptee object.	Inheritance — the adapter inherits from BOTH the target interface and the adaptee class at the same time.
Language support	Works in any OO language (Java, C#, Python, ...).	Only possible in languages that support multiple inheritance of classes (e.g., C++); not possible in Java, which only allows multiple interface inheritance.
Flexibility	Can adapt an adaptee AND any of its subclasses, since it only holds a reference (polymorphism works normally).	More rigid — tied to inheriting from one specific adaptee class.

Consequences

- Lets you introduce new adapters to integrate with existing code without breaking the code that already uses the target interface (Single Responsibility + Open/Closed).
- Trade-off: overall code complexity increases, since you must introduce a set of new interfaces and adapter classes; sometimes it is simpler to change the service class so it matches the rest of your code, if you have access to its source.

2.4 Bridge

Intent: Split a large class, or a set of closely related classes, into two separate, independent class hierarchies — Abstraction and Implementation — that can each be developed, extended, and varied independently of each other.

The problem in detail (Colored Shapes)

Suppose you have two shapes, Circle and Square, and you want to add color, Red and Blue. The naive approach creates one subclass per COMBINATION: RedCircle, BlueCircle, RedSquare, BlueSquare. As soon as you add a third color (Green) or a third shape (Triangle), the number of subclasses grows multiplicatively (shape-count × color-count), because the two properties — form and color — are actually two INDEPENDENT dimensions of variation, but inheritance forces them into a single, flat hierarchy that must enumerate every combination.

The solution

Recognize that you have two separate hierarchies — one for Shape (form) and one for Color — and connect them through composition instead of inheritance. A Shape object HOLDS a reference to a Color object and delegates the color-related work to it, rather than being subclassed per color. Now adding a new color means adding ONE new Color subclass (no changes to any Shape class), and adding a new shape means adding ONE new Shape subclass (which automatically works with every existing color, since it just holds a Color reference).

GoF terminology: Abstraction and Implementation

- Abstraction — the high-level control layer for some entity (also called the interface). This layer is not supposed to do real work on its own; it delegates the actual work to the implementation layer.
- Implementation — the low-level platform layer that does the real, concrete work (also called the platform).

Real-world application: a cross-platform GUI's Abstraction is the visible controls used by regular users and admins; its Implementation is a set of platform-specific APIs for Linux, Windows, and macOS. The Abstraction object controls the appearance/behavior of the app and delegates the actual rendering/OS work to whichever linked Implementation object it currently holds. Implementations are interchangeable at runtime, which is what enables the SAME GUI code to work under different operating systems.

Consequences

- Both hierarchies (Abstraction and Implementation) can be extended independently — you can add a new Abstraction subclass without touching Implementation classes, and vice versa.
- Reduces the total number of classes compared to a naive combinatorial inheritance hierarchy, and follows the Single Responsibility Principle, since Abstraction focuses on high-level logic and Implementation focuses on platform details.
- Trade-off: introduces an extra layer of indirection, which can make the code slightly harder to follow at first, and it only pays off when there really are two (or more) independent dimensions of variation.

2.5 Consequences summary — Structural Patterns

Note: Facade, Proxy, and Flyweight are also standard GoF structural patterns, but since they are not part of your syllabus, they are intentionally left out of these notes so you can focus on exactly what you'll be tested on: Decorator, Composite, Adapter, and Bridge.

Practice Questions — Structural Patterns

A. True/False:

Statement	T / F
1. The Decorator pattern defines a family of algorithms, encapsulates each one, and makes them interchangeable.	
2. The Composite pattern lets a client treat individual objects and compositions of objects uniformly.	
3. A Class Adapter uses composition, while an Object Adapter uses multiple inheritance.	
4. The Bridge pattern is useful when a class has two (or more) independent dimensions that would otherwise cause a combinatorial explosion of subclasses.	
5. The Decorator pattern lets you attach new behavior to an object dynamically by wrapping it, without altering other objects of the same class.	
6. In the Bridge pattern, the Abstraction layer does the real, concrete work while the Implementation layer only forwards calls.	

B. Define the following terminologies in brief:

Term	Definition (write yours)
Decorator	
Composite	
Adapter (Object vs Class)	
Bridge	
Abstraction (Bridge)	

C. Analyze the code below and determine which structural pattern it follows. Justify your answer by examining the class's responsibilities.

```
interface DataExporter {
    String exportData(String data);
}

class LegacyXmlExporter {                                // third-party class, cannot be modified
    public String toXml(String data) {
        return "<data>" + data + "</data>";
    }
}

class XmlExporterAdapter implements DataExporter {
    private LegacyXmlExporter legacyExporter;
    public XmlExporterAdapter(LegacyXmlExporter legacyExporter) {
        this.legacyExporter = legacyExporter;
    }
    public String exportData(String data) {
        return legacyExporter.toXml(data);                // translates the call
    }
}
```

D. For each scenario, identify the pattern and state the main reason (no more than 10 words):

1. A text editor needs to add bold, italic, and underline formatting to text in any combination, at runtime, without creating a class for every combination.
2. You want your legacy XmlParser class to work with a new library that only accepts JsonParser objects, without modifying either class.
3. A file-system explorer must display both single files and folders (which can contain more files and folders) using the exact same print()/getSize() calls.

3. Behavioral Design Patterns

Behavioral patterns are concerned with algorithms and the assignment of responsibilities between objects — that is, HOW objects communicate, collaborate, and hand off work to each other, and how the responsibility for carrying out a task flows through a system.

3.1 Strategy

Definition: Define a family of algorithms, encapsulate each one, and make them interchangeable. Strategy lets the algorithm vary independently from the clients that use it.

The problem in detail (Duck Simulator)

A simulator has many duck species. All ducks swim() and quack() the same way, but they fly() differently — and some ducks (RubberDuck, DecoyDuck) cannot fly or quack at all. A naive design puts fly() and quack() directly on an abstract Duck superclass and has each subclass override them as needed. This breaks down immediately: at a project demo, a RubberDuck was accidentally given real flying behavior because it inherited the default fly() and nobody overrode it — showing that inheriting behavior by default is fragile whenever some subclasses need to opt OUT of it.

The next instinct is to pull flying and quacking out into Flyable and Quackable interfaces, and have only the ducks that actually fly/quack implement them. This avoids giving RubberDuck a fly() method at all, but introduces a NEW problem: because interfaces have no shared implementation, every class that implements Flyable must write its OWN version of fly() — so if you need to tweak the flying algorithm even slightly (e.g., adjust wingbeat rate), you must go and edit EVERY class that implements Flyable separately. There is a lot of duplicated logic with no way to share it.

The design principle that unlocks the solution

Key principle

Identify the aspects of your application that vary and separate them from what stays the same. Program to an interface, not an implementation. Favor composition over inheritance.

Flying and quacking are the parts that VARY across duck species; swimming (and being a duck) is what stays the SAME. So: pull flying and quacking out into their own family of interchangeable behavior objects (FlyBehavior with implementations FlyWithWings and FlyNoWay; QuackBehavior with implementations Quack and MuteQuack), and give each Duck a REFERENCE to a FlyBehavior object and a QuackBehavior object, instead of implementing flying/quacking itself.

```
interface FlyBehavior { void fly(); }
class FlyWithWings implements FlyBehavior {
    public void fly() { System.out.println("I'm flying!!"); }
}
class FlyNoWay implements FlyBehavior {
    public void fly() { System.out.println("I can't fly."); }
}

abstract class Duck {
    FlyBehavior flyBehavior;           // HAS-A, not IS-A
    public void performFly() { flyBehavior.fly(); }
    public void setFlyBehavior(FlyBehavior fb) { flyBehavior = fb; }    // swap at runtime!
    public abstract void display();
}

class MallardDuck extends Duck {
    public MallardDuck() { flyBehavior = new FlyWithWings(); }
    public void display() { System.out.println("I'm a Mallard duck"); }
}
```

```
class RubberDuck extends Duck {
    public RubberDuck() { flyBehavior = new FlyNoWay(); }    // simply plug in the 'can't
fly' behavior
    public void display() { System.out.println("I'm a rubber duck"); }
}
```

Now fixing the flying ALGORITHM means editing exactly one class (e.g., FlyWithWings), and every duck that uses it is automatically fixed. Because setFlyBehavior() is a normal method, a Duck's flying behavior can even be swapped at RUNTIME — e.g., a duck could learn to fly, or a special "jet-pack" behavior could be attached to any duck on the fly. This runtime flexibility is a defining feature of Strategy.

Another example: navigation apps

A map application lets you switch between different route-calculation strategies — driving, walking, or public transit — in real time between two points. Each routing algorithm is encapsulated in its own strategy object implementing a common RouteStrategy interface (e.g., buildRoute(start, end)); the map's Navigator class simply holds a reference to the currently selected strategy and delegates to it, so changing the travel mode is just swapping which strategy object the Navigator points to.

Consequences

- Algorithms and the objects that use them can vary independently — you can add new strategies without touching the client class, and vice versa (Open/Closed Principle).
- Eliminates conditional statements that would otherwise be needed to choose behavior (e.g., avoids a big if/else on duck type inside fly()).
- Trade-off: if you only ever have two or three algorithms that rarely change, introducing the extra interfaces and classes may be more complexity than the problem warrants.

3.2 Template Method

Definition: Defines the SKELETON of an algorithm in a superclass, but lets subclasses override specific steps of the algorithm without changing the algorithm's overall structure/sequence.

The problem in detail

Consider preparing coffee and preparing tea. Both follow essentially the same skeleton: boil water → brew (steep tea leaves / brew coffee grounds) → pour into cup → add condiments (lemon for tea / sugar & milk for coffee). If you implement Coffee and Tea as two totally separate classes, you duplicate the "boil water" and "pour into cup" steps (which are IDENTICAL in both), and only the "brew" and "add condiments" steps actually differ. This duplication is exactly the kind of thing that should be shared.

The solution

Recognize that both recipes have the same "skeleton" — the same sequence of steps in the same order — and only some of the individual steps vary. Put the skeleton method in an abstract superclass, mark it FINAL so subclasses cannot change the overall sequence, and implement the steps that are the SAME directly in the superclass. For the steps that DIFFER, declare them as abstract methods that each subclass must implement in its own way.

```
abstract class CaffeineBeverage {
    // TEMPLATE METHOD -- the skeleton, marked final so subclasses can't reorder the steps
    final void prepareRecipe() {
        boilWater();
        brew();                // varies -> abstract
        pourInCup();
        if (customerWantsCondiments()) {    // a HOOK: optional step with a default
            addCondiments();                // varies -> abstract
        }
    }
}
```

```

    }
    void boilWater() { System.out.println("Boiling water"); } // common step
    void pourInCup() { System.out.println("Pouring into cup"); } // common step
    abstract void brew(); // subclass MUST implement
    abstract void addCondiments(); // subclass MUST implement
    boolean customerWantsCondiments() { return true; } // HOOK -- subclasses MAY override
}

class Coffee extends CaffeineBeverage {
    void brew() { System.out.println("Dripping coffee through filter"); }
    void addCondiments() { System.out.println("Adding sugar and milk"); }
}
class Tea extends CaffeineBeverage {
    void brew() { System.out.println("Steeping the tea"); }
    void addCondiments() { System.out.println("Adding lemon"); }
}

```

A HOOK is a method with a default (often empty, or default-true) implementation in the abstract class that a subclass MAY override to plug into the algorithm at a specific point, without being FORCED to (unlike an abstract method, which every subclass MUST implement). Hooks give subclasses limited, optional control over the algorithm's behavior, e.g., letting TeaWithHook decide it does NOT want condiments by overriding customerWantsCondiments() to return false based on user input.

Difference between Strategy and Template Method — a very common exam question

Aspect	Template Method	Strategy
Mechanism	Based on inheritance — alters parts of an algorithm by having subclasses override specific steps.	Based on composition — alters the whole algorithm/behavior by supplying the context object with a different strategy OBJECT.
When behavior is fixed	At compile time / class-definition time — you must create a new subclass to get new behavior. It works at the class level, so it's static.	At runtime — you can call setStrategy() to swap the algorithm object while the program is running. It works at the object level.
What varies	Only specific STEPS of an otherwise-fixed algorithm skeleton.	The ENTIRE algorithm is swappable as one unit.

3.3 Observer

Definition: Define a one-to-many dependency between objects so that when ONE object (the Subject) changes state, ALL of its dependents (the Observers) are notified and updated automatically.

The problem in detail (Weather Station)

A WeatherData object receives new temperature/humidity/pressure readings from a physical weather station. Several different displays (CurrentConditionsDisplay, StatisticsDisplay, ForecastDisplay) must all update themselves whenever new data arrives, and the set of displays must be able to grow or shrink at runtime (new displays added, old ones removed) without changing WeatherData's code. A naive first implementation has WeatherData directly call each specific display's update method by name inside its measurementsChanged() method — e.g., `currentConditionsDisplay.update(...); statisticsDisplay.update(...);` — hard-coded one by one. This tightly couples WeatherData to every CONCRETE display class, and adding or removing a display means editing WeatherData's source code directly, which is exactly the kind of change the Open/Closed Principle warns against.

The solution — like a magazine subscription

This is exactly how a magazine subscription (publish/subscribe) works: a publisher doesn't need to know in advance who its subscribers are, or how many there will be — anyone can subscribe or unsubscribe at any time, and the publisher just sends the same update to whoever is currently subscribed. WeatherData becomes the Subject/Publisher: it maintains a LIST of Observer references and exposes registerObserver(o), removeObserver(o), and notifyObservers(). Each display becomes an Observer/Subscriber implementing a common update() method. WeatherData's code never needs to know the CONCRETE class of any observer — it just loops over its list and calls update() on each one.

```
interface Observer { void update(float temp, float humidity, float pressure); }
interface Subject {
    void registerObserver(Observer o);
    void removeObserver(Observer o);
    void notifyObservers();
}

class WeatherData implements Subject {
    private List<Observer> observers = new ArrayList<>();
    private float temperature, humidity, pressure;

    public void registerObserver(Observer o) { observers.add(o); }
    public void removeObserver(Observer o) { observers.remove(o); }
    public void notifyObservers() {
        for (Observer o : observers) o.update(temperature, humidity, pressure);
    }
    public void measurementsChanged() { notifyObservers(); } // called when new data
    arrives
}

class CurrentConditionsDisplay implements Observer {
    private Subject weatherData;
    public CurrentConditionsDisplay(Subject weatherData) {
        this.weatherData = weatherData;
        weatherData.registerObserver(this); // subscribes itself in its own
    constructor
    }
    public void update(float t, float h, float p) {
        System.out.println("Current conditions: " + t + "F and " + h + "% humidity");
    }
}
```

Note the constructor of CurrentConditionsDisplay is passed a reference to the Subject and immediately registers itself as an observer of it — this is the standard idiom that keeps the coupling one-directional: the concrete display knows about the Subject interface, but WeatherData never needs to know about any concrete display class.

Terminology note

In more modern (and GoF-adjacent) terminology, Subject = Publisher, and Observer = Subscriber. Both terms mean the same thing and appear interchangeably in different textbooks and exam questions.

Consequences

- Subject and Observers are loosely coupled — Subject only depends on the Observer INTERFACE, never on concrete observer classes.
- Observers can be added or removed at runtime, supporting dynamic subscription.
- Trade-off: if there are many observers, or if update() triggers further changes that trigger more notifications, you can end up with unpredictable update orders or even cascading update chains that are hard to debug.

3.4 Mediator

Definition: Restricts direct communication between a set of objects and forces them to collaborate only via a mediator object, reducing chaotic, many-to-many dependencies between them.

The problem in detail (UI Dialog)

Consider a customer-profile dialog box with many interacting form controls: a checkbox ("Business account?"), a text field (company name, enabled only if the checkbox is checked), and an "Apply" button that must validate every field before submitting. A naive design puts the interaction logic directly inside each control's own class — e.g., the checkbox's click handler directly references and enables/disables the specific text field object. This tightly couples the checkbox class to one specific button, one specific text field, and one specific dialog. The immediate consequence: that checkbox class can no longer be reused in ANY other dialog, because it has hard-coded references to the components of this ONE dialog.

This is a real-world analogy familiar from air traffic control: pilots landing at an airport don't talk directly to every other aircraft in the sky — all communication is routed through the control tower. Without the tower, every pilot would need to be aware of every other aircraft, and coordination would become impossibly complex (and dangerous) as the number of aircraft grows.

The solution

Cease all direct communication between the components. Instead, each component only talks to a single MEDIATOR object (in the UI example, the Dialog itself), which redirects calls to whichever other component(s) actually need to react. The Dialog (mediator) knows about ALL of its sub-elements, so no NEW dependency is introduced by adding the mediator — but critically, the checkbox no longer needs to know about the text field directly. Instead, the checkbox just notifies the mediator ("I was toggled"), and the mediator decides what to do (enable or disable the text field). Similarly, the "Apply" button just notifies the mediator that it was clicked; it does not need to know who else exists in the dialog.

```
interface DialogMediator {
    void notify(Component sender, String event);
}

class ProfileDialog implements DialogMediator {
    private Checkbox businessCheckbox;
    private TextField companyField;
    private Button applyButton;
    // constructor wires up components and passes 'this' as their mediator

    public void notify(Component sender, String event) {
        if (sender == businessCheckbox && event.equals("checked")) {
            companyField.setEnabled(businessCheckbox.isChecked());
        }
        if (sender == applyButton && event.equals("click")) {
            validateAndSubmit();
        }
    }
}
```

Components now depend only on a single, abstract mediator interface, instead of being coupled to dozens of their sibling components — the checkbox, text field, and button classes can now each be reused in a completely different dialog, as long as that new dialog implements the DialogMediator interface itself.

Difference between Mediator and Observer

Aspect	Mediator	Observer
Primary goal	Eliminates mutual, tangled dependencies between MANY	Establishes a dynamic, one-way, one-to-many

	objects by routing everything through a single central object.	subscribe/unsubscribe relationship between exactly one subject and many observers.
Structure	A hub-and-spoke structure with one mediator and many colleague objects that all depend on it.	No central hub required beyond the subject itself; observers can be added/removed freely.

3.5 State

Definition: Allow an object to alter its behavior when its internal state changes. The object will appear to change its class.

The problem in detail (Gumball Machine)

A gumball machine can be in one of several states — NoQuarter, HasQuarter, Sold, SoldOut (and later, an added Winner state for a "10% chance of a free gumball" promotion). The actions insertQuarter(), ejectQuarter(), turnCrank(), and dispense() behave DIFFERENTLY depending on which state the machine is currently in — e.g., turnCrank() should only actually dispense a gumball if a quarter has already been inserted. A first attempt encodes all of this using an integer/enum "state code" field and a big if/else or switch inside EVERY action method, checking the current state code and branching accordingly. This works initially, but every time a new state or a new business rule is introduced (like the Winner state), you must find and edit EVERY SINGLE action method's conditional logic — Fowler would call this a Divergent Change / Shotgun Surgery smell, and the slides call it "the nightmare of change."

The solution

Extract EACH state into its OWN class implementing a common State interface with one method per possible action (insertQuarter(), ejectQuarter(), turnCrank(), dispense()). The GumballMachine (the "context") holds a reference to its CURRENT State object and simply DELEGATES every action call to that object — the machine itself no longer contains any state-dependent conditional logic. Each concrete state class implements only the behavior that makes sense for that state (e.g., NoQuarterState.insertQuarter() transitions to HasQuarterState; NoQuarterState.turnCrank() just prints an error, since there's no quarter yet), and each state object is responsible for calling back into the machine to trigger the transition to the NEXT state object when appropriate.

```
interface State {
    void insertQuarter();
    void ejectQuarter();
    void turnCrank();
    void dispense();
}

class NoQuarterState implements State {
    GumballMachine machine;           // reference back to the context
    public NoQuarterState(GumballMachine m) { machine = m; }
    public void insertQuarter() {
        System.out.println("You inserted a quarter");
        machine.setState(machine.getHasQuarterState()); // transition!
    }
    public void ejectQuarter() { System.out.println("You haven't inserted a quarter"); }
    public void turnCrank()    { System.out.println("You turned, but there's no
quarter"); }
    public void dispense()     { System.out.println("You need to pay first"); }
}

class GumballMachine {
    State noQuarterState, hasQuarterState, soldState, soldOutState;
    State currentState;
    public void setState(State s) { currentState = s; }
```

```

    public void insertQuarter() { currentState.insertQuarter(); } // pure delegation
    public void turnCrank()      { currentState.turnCrank(); }
    // ... getters for each state field ...
}

```

Adding the new Winner state now means writing ONE new class (WinnerState) and updating the states that need to be aware it might be entered (e.g., HasQuarterState.turnCrank() randomly transitions to either SoldState or WinnerState) — the OTHER, unrelated state classes do not need to be touched at all.

Difference between State and Strategy — a very common exam question

State and Strategy are structurally almost identical ("twins separated at birth") — both are based on composition and delegate behavior to a swappable helper object. The difference is in intent and in how much the helper objects are allowed to know about each other:

Aspect	State	Strategy
Awareness between helper objects	Concrete states are typically AWARE of each other and are allowed to trigger transitions to one another (e.g., NoQuarterState knows about HasQuarterState).	Concrete strategies are made completely INDEPENDENT of, and unaware of, one another — swapping one strategy for another never involves one strategy object talking to another.
Who changes the current object	The state objects themselves usually decide when and how to transition the context to a new state.	Only the CLIENT (external code) decides to swap the context's current strategy; a strategy object never swaps itself out.
Purpose	Model an object whose behavior changes as its internal condition changes over time (a finite-state machine).	Offer several interchangeable ways to perform ONE conceptual operation, chosen based on external criteria.

3.6 Iterator (bonus — not covered in slides, but useful to know)

Intent: Provide a way to access the elements of a collection (a list, tree, graph, etc.) sequentially, one at a time, WITHOUT exposing the collection's underlying internal representation (array, linked list, tree structure, etc.) to the code that is iterating.

Reasoning: without Iterator, code that wants to loop over a collection needs to know its exact internal structure (e.g., manually managing an array index, or walking tree-node pointers). This couples client code to that structure, and makes it hard to later swap out the internal representation. An Iterator object encapsulates the current position and exposes just hasNext()/next(), so the client can loop uniformly regardless of what's happening underneath (this is exactly what Java's `for (Item i : collection)` loop uses behind the scenes).

Note

Facade, Proxy, Flyweight, Command, Chain of Responsibility, Visitor, and Memento are also standard GoF patterns, but since they are not part of your syllabus, they are intentionally left out of these notes. This section only keeps Iterator as a brief bonus, since it's a small, easy concept — but the focus for your exam should stay on the six patterns covered above: Strategy, Template Method, Observer, Mediator, and State (plus Iterator here as an optional extra).

Practice Questions — Behavioral Patterns

A. True/False:

Statement	T / F
-----------	-------

1. The Strategy pattern defines a family of algorithms, encapsulates each one, and makes them interchangeable.	
2. Template Method works at the object level and lets you swap behavior at runtime, while Strategy works at the class level.	
3. The Observer pattern defines a one-to-many dependency so that when the subject changes state, all dependents are notified automatically.	
4. In the Mediator pattern, colleague objects communicate directly with each other instead of through a central object.	
5. State and Strategy are both based on composition, but Strategy keeps its behavior objects unaware of each other while State does not.	
6. The Iterator pattern exposes a collection's internal representation so that client code can loop over it directly.	
7. A Template Method is typically declared final so that subclasses cannot change the overall order of steps in the algorithm.	

B. Define the following terminologies in brief:

Term	Definition (write yours)
Observer	
Mediator	
State	
Strategy	
Template Method Hook	

C. Scenario — identify the pattern(s) and implement the solution.

Imagine a ride-sharing app where a Trip can be in one of these states: Requested, Ongoing, Completed, or Cancelled. What the app allows you to do (cancel, start, complete) depends on the trip's current state. Separately, the app lets the rider choose a fare-calculation strategy — Standard, Surge, or Discounted — and how fares are computed changes independently of the trip's state. Name the pattern(s) you would use and sketch a class diagram or code skeleton.

D. Short answer:

1. Explain, with an example, the difference between the Observer and Mediator patterns.
2. Explain, with an example, the difference between the Template Method and Strategy patterns.
3. Explain, with an example, why State and Strategy are considered "twins separated at birth," and what actually distinguishes them.

4. Git & Version Control

4.1 What is version control, and why does it exist?

Version control software exists because "the goal" of good software engineering is to never write the same code twice, and because writing code is hard — losing work, or being unable to go back to an earlier working version, is extremely costly. Before dedicated version-control tools, developers sometimes tried to manage "versions" of a project manually — folders named things like "Really important project - Copy (2) - Restored - FINAL - BACKUP" — which is unreliable, doesn't scale to teams, and provides none of the guarantees a real system does.

A proper version control system maintains the FULL HISTORY of every change ever made, lets you travel between different branches representing different in-progress features, and lets you compare and combine different versions of the code (merging work from multiple people or multiple lines of development back together).

4.2 Centralized vs. Distributed version control

Older systems like Subversion (SVN) are CENTRALIZED: there is one central server holding the full version database/history, and each individual computer only checks out (downloads) the current working copy of files — NOT the full history. If the central server goes down, or a network connection is unavailable, nobody can view history, commit, or branch.

Git is a form of DISTRIBUTED version control: EVERY computer that clones a Git repository gets a complete copy of the ENTIRE project, including its full history, stored locally on that machine. This means you can commit, branch, view history, and do almost everything else completely offline, and there is no single point of failure — any clone can, in principle, restore the whole project.

4.3 Git vs. GitHub — a very common point of confusion

- GitHub is a WEBSITE for sharing and collaborating on Git repositories (it provides hosting, pull requests, issue tracking, etc.).
- You do NOT need GitHub to use Git — Git itself runs entirely locally, on your own machine, and works perfectly well with no internet connection and no GitHub account at all. GitHub is simply one popular place to host a shared/remote copy of a Git repository (GitLab and Bitbucket are other examples).

4.4 Installing and starting Git

- Download Git from <https://git-scm.com/>
- Git can be used from a terminal — on Windows, the "Git Bash" desktop application provides a Unix-like terminal for this purpose.
- Every Git command starts with the word ``git`` (e.g., ``git status``).
- Get help for any specific command with ``git help [command name]``.

4.5 The Three Areas — the mental model you need for everything else

Nearly every Git command makes more sense once you understand these three distinct "areas" that your files pass through:

- Working Directory — the actual files you see and edit on disk, in your normal file browser/editor.
- Staging Area (also called the "index") — a TEMPORARY holding area where you prepare exactly which changes will go into the NEXT commit. Changes are moved here with ``git add``. This exists so that you

can choose to commit only SOME of your current changes, leaving others uncommitted for later, rather than being forced to commit everything you've touched at once.

- `.git` directory (the Repository) — created by ``git init``, this stores the ENTIRE project history (every past commit) plus additional state, such as which branch/commit you currently have checked out.

How they connect

Checkout moves a snapshot FROM the `.git` repository INTO your Working Directory. ``git add`` ("Stage Fixes") moves your edits FROM the Working Directory INTO the Staging Area. ``git commit`` moves the staged snapshot FROM the Staging Area permanently INTO the `.git` repository as a new commit.

4.6 Core commands, one by one, in depth

git init

Creates a brand-new, EMPTY Git repository in the current directory by creating a hidden ``.git`` folder. This folder is what actually stores the entire project's history and additional Git state (such as which branch/commit you are currently viewing) — deleting the ``.git`` folder effectively destroys all Git history for that project (though your working files remain untouched).

git status

Displays a summary of Git's current understanding of your project: which files have been modified but not yet staged, which files are staged and ready to be committed, and which branch you are currently on. This is the single most useful "what's going on right now?" command, and is safe to run at any time since it never changes anything.

git add

Stages changes — that is, marks specific changes as ready to be included in the NEXT commit.

```
git add path/to/file      # stage one specific file's changes
git add -A                # stage ALL changes in the repository (new, modified, and deleted files)
```

git commit

Creates a new commit — a permanent SNAPSHOT of the working directory as it currently exists in the staging area, along with a commit message describing the change.

```
git commit                # opens an editor to write a commit message, then commits staged changes
git commit -m "message"   # supply the commit message directly on the command line
git commit -a              # stage ALL currently-tracked, modified files AND commit, in one step
git commit -a -m "message" # combine both shortcuts
git commit --amend         # edit/replace the MOST RECENT commit (e.g., to fix a typo in its message
                           # or add a forgotten file), instead of creating a brand-new commit
```

The standard editing workflow, repeated over and over during development, is: (1) edit or create some files; (2) stage the changes you want in this commit with ``git add``; (3) commit the staged changes with ``git commit``; (4) write a commit message and save; (5) repeat.

git diff

Shows exactly WHICH LINES have changed since your last commit (i.e., differences between your working directory/staging area and the most recent commit) — by default this does NOT compare against any remote branch at all, only against your own local history. A very common exam trap is assuming ``git diff`` compares against a remote branch by default; it does not.

```
git diff          # show all changes since the last commit
git diff path/to/file  # scope the comparison to just one specific file
```

git log

Shows a chronological list of every action (commit) Git has recorded — essentially "a timeline for your timeline." It lists each commit's unique hash and its commit message, letting you scroll back through the project's entire history.

git checkout

A surprisingly multi-purpose command, whose meaning depends on what you pass it:

- ``git checkout [commit hash] path/to/file`` — copies ONE specific file, as it existed in that specific commit, into your current working directory (this **OVERWRITES** the existing copy of that file). You can use a shortened PREFIX of the full commit hash, as long as it's unambiguous.
- ``git checkout path/to/file`` — copies that file from the **MOST RECENT** commit into the working directory (useful for discarding your own uncommitted local edits to that one file).
- ``git checkout [branch name]`` — switches your working directory to reflect a different branch.
- ``git checkout [commit hash]`` (with no file given) — copies the **ENTIRE** commit's snapshot into your working directory, but this puts you into a special "detached HEAD" state (explained below), since you are no longer sitting on any named branch.

4.7 Commits, in depth

- A commit is a **SNAPSHOT** of your entire working directory at one moment in time — specifically, it records the state of all **TRACKED** files (files Git already knows about, as opposed to new, never-added files).
- Every commit is identified by a **HASH** — a long, deterministically-generated, effectively-unique identifier (Git uses SHA-1 hashes). Two commits with identical content and history will always get the exact same hash, and any change at all to the content produces a completely different hash.
- Each commit stores a reference (a pointer) to the commit(s) that came immediately before it, which is how Git reconstructs the **FULL** history just by "walking backward" from any given commit. A commit can have more than one "parent" commit — this happens specifically at merge commits, where two separate lines of history are joined back together.
- Commits are, by convention, almost **NEVER** edited or deleted after the fact (with the narrow exception of ``git commit --amend`` on the most recent, not-yet-shared commit) — this immutability is what makes the history trustworthy.

4.8 Branches, in depth

The full commit history of a project is technically a **Directed Acyclic Graph (DAG)**: each commit points "backward" to its parent(s), edges only ever point one direction (toward the past), and there are never any cycles (a commit can never eventually point back to itself).

A common point of confusion: a commit graph with many branching lines does **NOT** necessarily mean there are multiple "branches" in the Git sense at all — a branch is simply a **MOVABLE POINTER** (a named label) that refers to one specific commit. Looking at just a raw commit graph, with no labels attached, there are technically zero "branches" — branches only exist once you actually create named pointers like ``master``, ``feature1``, etc., which each point at one particular commit in that graph.

- **HEAD** is Git's special pointer to whichever branch (or, in a detached state, whichever specific commit) you currently have checked out — it answers the question "where am I right now?"

- When you make a NEW commit while sitting on a branch, that branch's pointer automatically moves FORWARD to point at the new commit, and HEAD (which points at the branch) effectively moves forward along with it.

Branch commands

```
git branch [branch name]    # create a new branch (pointer), but do NOT switch to it
git branch                  # list all existing branches in this repository
git branch -d [branch name] # delete a branch (only if it has already been merged)
git checkout [branch name]  # switch your working directory + HEAD to point at this branch
git checkout -b [branch name] # create a new branch AND immediately switch to it, in one step
```

Detached HEAD state

A detached HEAD occurs specifically when you run ``git checkout [commit hash]`` (rather than a branch name) — HEAD then points DIRECTLY at that one commit, instead of pointing at any named branch. This is a common source of confusion and lost work for beginners: if you make new commits while in a detached HEAD state, those new commits are NOT attached to any branch, so as soon as you check out a different branch, those new commits become effectively unreachable/orphaned (and will eventually be garbage-collected by Git) unless you took action first.

The simple, standard fix: run ``git checkout -b [new branch name]`` WHILE still in the detached HEAD state (before switching away). This creates a brand-new branch pointer at your CURRENT commit, effectively "rescuing" that commit (and any new commits you made) by giving it a permanent home on a named branch.

4.9 Merging

``git merge [branch name]`` merges the changes from the NAMED branch/commit INTO your CURRENT branch (the one you have checked out). Git determines exactly what changed on each side by finding the LEAST COMMON ANCESTOR of the two branches (the most recent commit that both branches share in their history) and comparing each branch's changes against that shared ancestor.

The standard merge workflow, step by step:

1. Checkout the branch you want to merge INTO (i.e., the branch that should end up containing the combined result).
2. Run ``git merge [other-branch-name]`` for the branch whose changes you want to bring in.
3. Run ``git status`` to check whether there are any merge conflicts (places where both branches changed the SAME lines of the SAME file in different, incompatible ways — Git cannot automatically decide which version to keep).
4. If there ARE conflicts, manually edit the conflicted files to resolve them (choosing/combining the correct content), then ``git add`` each resolved file to mark it as fixed.
5. Run ``git commit`` to finalize the merge as a new "merge commit" (which has two parent commits — one from each side that was merged).

4.10 git revert and git stash

- ``git revert [commit hash]`` undoes the EFFECT of a specific past commit by creating a brand-new commit that applies the OPPOSITE change. This is considered the SAFE way to undo something that has already been shared/pushed with others, because it never rewrites or deletes existing history — it only adds a new commit on top, so nobody else's copy of the history becomes inconsistent.
- ``git stash`` temporarily shelves (sets aside) your current UNCOMMITTED changes, restoring your working directory to match the last commit, so you can cleanly switch branches, pull updates, or otherwise work

with a clean working directory. Very important exam point: stashed changes remain PURELY LOCAL to your own machine — they are NOT automatically transferred to any remote repository, and running ``git push`` does NOT push stashed changes at all (only actual commits get pushed). To get your stashed work back, you must explicitly run ``git stash pop`` (which re-applies the stash and removes it from the stash list) or ``git stash apply`` (which re-applies it but keeps a copy in the stash list) — and then still ``git add`` + ``git commit`` + ``git push`` as usual if you want that work to be shared.

4.11 .gitignore

`.gitignore` is a plain-text file, placed in your repository, that lists file/directory PATTERNS Git should NEVER track at all — for example, compiled build output, IDE configuration folders, local secrets/credentials, or dependency folders like `node_modules`. It is important to understand precisely what this does and does NOT do: it tells Git to completely IGNORE matching files (they will never show up in ``git status``, can never be accidentally staged with a blanket ``git add -A``, and are never part of any commit). It does NOT "track uncommitted files" in any sense — quite the opposite: it prevents files from ever being tracked in the first place.

4.12 Moving edits between branches — a very common practical exam question

Scenario A: you accidentally edited ``main`` but have NOT yet committed the change, and you want that change to live on ``dev`` instead.

```
# Option 1: stash and move (safest, general-purpose approach)
git stash                # shelve the uncommitted edits
git checkout dev         # switch to the target branch
git stash pop            # re-apply the edits here
git add .
git commit -m "message"
```

Scenario B: you accidentally edited ``main`` AND ALREADY COMMITTED the change on ``main``, and now want that commit moved onto ``dev`` and removed from ``main``.

```
git checkout dev
git cherry-pick main     # copy that one commit's changes onto dev as a new commit
git checkout main
git reset --hard HEAD~1  # remove the mistaken commit from main (DANGEROUS: discards it there)
```

``git cherry-pick [commit]`` applies the changes introduced by ONE specific commit from elsewhere onto your current branch, as a brand-new commit — it is the standard tool for moving a single commit between branches without merging everything else.

4.13 Dissecting a Web Address

This topic bridges Git/general web knowledge and the JMeter topic below, since JMeter's HTTP samplers are configured using exactly these same pieces. Example: `https://www.google.com/imghp?hl=en&tab=ri`

- Protocol: `https` — the scheme used to communicate with the server (`http`, `https`, `ftp`, etc.).
- Server Name: `www.google.com` — the hostname / domain of the server being contacted.
- Path: `/imghp` — the specific resource being requested on that server.
- Parameter Name / Value pairs: everything after the ``?`` is the QUERY STRING, and individual parameters are separated by ``&``, each in the form ``name=value``. Here: `hl = en`, `tab = ri`.

General pattern to memorize: ``protocol://server-name/path?`

`param1=value1¶m2=value2¶m3=value3...`` — this exact structure recurs in both the Git/web-fundamentals topic and the JMeter topic's own worked examples.

A. True/False:

Statement	T / F
1. By default, git diff shows differences between the working directory and the remote branch of the repository.	
2. The git add command directly records changes to the repository by creating a new commit.	
3. The git stash command transfers stashed changes to the remote repository when the next git push is executed.	
4. The .gitignore file is used to track all uncommitted files in a repository.	
5. git checkout -b newbranch both creates a new branch and switches to it.	
6. A detached HEAD state means HEAD is pointing directly at a commit rather than at a branch.	
7. git revert is considered safer than rewriting history because it adds a new commit instead of deleting an existing one.	
8. Git is a centralized version control system where only the server stores the complete project history.	

B. Define the following terminologies in brief:

Term	Definition (write yours)
Staging Area	
Commit	
Branch	
Detached HEAD	
Merge Conflict	
Cherry-pick	

C. The current repository has two branches, main and feature. You have mistakenly edited main and already committed the change on main. Write the commands to move that commit to feature and remove it from main.

D. Find the protocol, server name, path, parameter name(s), and parameter value(s) from:
<https://cse.buet.ac.bd/portal/results?student=190101&sem=7>

5. Clean Coding Practices

"Any fool can write code that a computer can understand. Good programmers write code that humans can understand." This is the founding idea of clean code: code is read FAR more often than it is written — by your teammates, by your future self, by whoever maintains the system after you move on — so optimizing for HUMAN readability, not just for "it compiles and works," pays off enormously over a project's lifetime.

5.1 Why clean code matters

- Readability — code that reads almost like well-organized prose is far faster for anyone (including you, months later) to understand.
- Maintainability — clean code is easier and safer to change, since its structure and intent are clear, reducing the chance that a small fix accidentally breaks something unrelated.
- Collaboration — in a team, consistent, clear code lets multiple people work on the same codebase without constantly having to reverse-engineer each other's intentions.
- Bug reduction — code that clearly expresses its intent makes it much easier to spot logic errors, both for the original author and for reviewers.
- Efficiency — over the FULL lifetime of a project, time spent writing clean code up front is repaid many times over in reduced debugging and onboarding time (this is efficiency of the overall process, not necessarily raw CPU efficiency).

Key factors underlying all of the above: Effectiveness (does it do the right thing correctly), Efficiency (is it reasonably economical in resources), and Simplicity (is it as straightforward as it can be, without unnecessary cleverness or complexity).

5.2 Principle 1 — Format and Syntax

Keep formatting CONSISTENT throughout a codebase, and follow whatever standard convention your language/team uses (indentation width, brace placement, spacing around operators). Consistency matters more than which specific style is chosen — a codebase that mixes styles randomly is much harder to scan visually than one that picks a single style and sticks to it everywhere.

```
// BAD -- inconsistent/cramped spacing and indentation
const myFunc=(number1,number2)=>{
  const result=number1+number2;
  return result;
}

// GOOD -- consistent spacing, clear indentation
const myFunc = (number1, number2) => {
  const result = number1 + number2
  return result
}
```

5.3 Principle 2 — Naming Conventions

Use MEANINGFUL, SELF-EXPLANATORY names for variables, functions, and classes. A good name should tell the reader WHAT something is or does without requiring them to read the implementation, or a comment, to find out.

```
// BAD -- what do fxn, a, n, x mean? You have to read the body to find out.
int fxn(int a[], int n) {
  int i, x = 0;
  for (i = 0; i < n; i++) {
    if (a[i] > x) { x = a[i]; }
  }
}
```

```

    return x;
}

// GOOD -- the name and parameters tell you exactly what's happening
int findLargestElement(int array[], int size) {
    int i, largest = array[0];
    for (i = 1; i < size; i++) {
        if (array[i] > largest) { largest = array[i]; }
    }
    return largest;
}

```

Note that the GOOD version also fixed a subtle bug (starting the loop at `i=1` and initializing `largest` to `array[0]`, rather than starting at `i=0` with `largest=0`, which fails if all array values are negative) — this illustrates how clearer code also tends to surface hidden bugs more easily, since sloppy variable names can hide sloppy logic.

5.4 Principle 3 — Comments: when they help and when they hurt

Comments are usually written because the author senses that the code ISN'T intuitive on its own. In that specific situation, a comment can act like "deodorant masking the smell of fishy code" — it treats the SYMPTOM (confusing code) rather than the CAUSE (poorly named methods/variables). The single best comment is often simply a GOOD NAME for a method or variable, because a good name is documentation that can never go out of sync with the code (whereas a comment can easily become wrong/stale after the code beneath it changes, since nothing forces developers to update comments when they update logic).

```

// BAD -- comments compensate for meaningless names
void proc(Order o) {
    // check if order is valid
    if (o.items == null || o.amt < 0) throw new Exception("bad");
    // apply 10% discount if eligible
    if (o.loyalty) o.amt *= 0.9;
}

// GOOD -- the method names ARE the comments; no deodorant needed
void processOrder(Order order) {
    validateOrder(order);
    applyLoyaltyDiscount(order);
}
void validateOrder(Order order) {
    if (order.items == null || order.amt < 0) throw new Exception("Invalid order");
}

```

This does NOT mean comments are always bad. Comments genuinely ARE useful in two situations:

- To explain WHY something is done a certain (perhaps non-obvious) way — a name can describe WHAT the code does, but often cannot capture WHY a particular constraint exists. Example: `// Stripe rejects amounts > 999999. Cap prevents silent API failure.` — no method name alone could convey that business/technical context.
- To explain a genuinely COMPLEX ALGORITHM that isn't self-evident from reading the code, especially when referencing a well-known named technique. Example: `// Luhn algorithm: validates card numbers by doubling every 2nd digit from the right.` — naming the algorithm gives the reader a starting point to look up the full technique elsewhere, rather than reverse-engineering it line by line.

5.5 Principle 4 — SRP (Single Responsibility Principle)

A function (or class) should do ONE thing, and do it well. Keep functions and methods concise and focused. When a function accumulates multiple unrelated responsibilities, it becomes harder to name accurately (a sign of the problem), harder to test in isolation, and harder to reuse individual pieces of its logic elsewhere.

```

// BAD -- one function doing validation, business logic, formatting, AND printing

```

```

public void printGrade(String name, int score) {
    if (score < 0 || score > 100)
        throw new IllegalArgumentException("Score must be 0-100");
    String grade = score >= 90 ? "A" : score >= 75 ? "B" : score >= 60 ? "C" : "F";
    String result = String.format("Student: %-15s Score: %d Grade: %s", name, score,
grade);
    System.out.println(result);
}

// GOOD -- each function has exactly ONE reason to change
public void validate(int score) {
    if (score < 0 || score > 100)
        throw new IllegalArgumentException("Score must be 0-100");
}
public String toLetterGrade(int score) {
    return score >= 90 ? "A" : score >= 75 ? "B" : score >= 60 ? "C" : "F";
}
public String formatGrade(String name, int score, String grade) {
    return String.format("Student: %-15s Score: %d Grade: %s", name, score, grade);
}
public void printGrade(String name, int score, String grade) {
    System.out.println(formatGrade(name, score, grade));
}
// Caller orchestrates each responsibility explicitly:
public void handleGrade(String name, int score) {
    validate(score);
    String grade = toLetterGrade(score);
    printGrade(name, score, grade);
}
}

```

Now, if the grading SCALE changes, only `toLetterGrade()` needs to change. If the DISPLAY format changes, only `formatGrade()/printGrade()` need to change. Each function can also be independently unit-tested (e.g., you can test `toLetterGrade(85)` returns "B" without needing to print anything or care about validation logic at all).

5.6 Principle 5 — DRY (Don't Repeat Yourself)

The same logic — especially business rules or validation logic — should not be copy-pasted into multiple places. When knowledge exists in only ONE place in the codebase, a change to that knowledge only requires ONE edit; when it is duplicated, every future change requires you to remember to find and update EVERY copy, and it is very easy to update some copies and forget others, leading to inconsistent behavior.

```

// BAD -- the SAME email-validation rule is copy-pasted into three different classes
class RegistrationService {
    void register(String email) {
        if (!email.contains("@") || email.length() < 5)    // copy 1
            throw new IllegalArgumentException("Invalid email");
    }
}
class ProfileUpdateService {
    void updateEmail(String email) {
        if (!email.contains("@") || email.length() < 5)    // copy 2
            throw new IllegalArgumentException("Invalid email");
    }
}
class AdminUserService {
    void createAdmin(String email) {
        if (!email.contains("@") || email.length() < 5)    // copy 3
            throw new IllegalArgumentException("Invalid email");
    }
}

// GOOD -- the validation rule lives in exactly ONE class
class EmailValidator {

```

```

    public boolean isValid(String email) {
        if (email == null) return false;
        if (!email.contains("@")) return false;
        if (email.length() < 5) return false;
        // New rule added HERE ONCE -- applies everywhere automatically:
        if (!email.endsWith(".com") && !email.endsWith(".org")) return false;
        return true;
    }
}
class RegistrationService {
    private final EmailValidator emailValidator;
    void register(String email) {
        if (!emailValidator.isValid(email))
            throw new IllegalArgumentException("Invalid email");
    }
}

```

Notice how adding a brand-new validation rule (checking the domain suffix) now requires editing `EmailValidator` **ONCE**, and it automatically applies to registration, profile updates, and admin creation — without touching those three classes at all.

5.7 Principle 6 — Conciseness vs. Clarity

A very compact, clever one-liner is not automatically "better" code — if it takes the reader significantly longer to decode than a slightly longer, well-structured version, the extra conciseness has a real cost. Prefer clarity over cleverness whenever the two trade off against each other.

```

// Example 1: concise -- fast to write, slower to read for many people
const countVowels = s => (s.match(/[aeiou]/gi) || []).length;
console.log(countVowels("hello world"));

// Example 2: more verbose, but each named step is self-explanatory
function countVowels(s) {
    const vowelRegex = /[aeiou]/gi;
    const matches = s.match(vowelRegex) || [];
    return matches.length;
}
console.log(countVowels("hello world"));

```

Both versions are equally **CORRECT** — the question is purely about how quickly a new reader (including future you) can understand what's happening. The named `vowelRegex`` variable in Example 2 also acts as inline self-documentation, similar in spirit to the naming-conventions principle above.

5.8 Principle 7 — Reusability

This principle is closely related to, and largely a consequence of, both SRP and DRY: functions and classes that each do exactly **ONE** well-defined thing, and that avoid duplicating logic that exists elsewhere, naturally end up being easy to **REUSE** in new contexts, because they aren't tangled up with unrelated responsibilities. The `EmailValidator` example above is a good illustration — because it does exactly one thing well, it can be reused by any future service that also needs to validate an email address, with zero extra work.

5.9 Principle 8 — Single Source of Truth

Avoid **HARD-CODING** numbers ("magic numbers") or other configuration values directly inside business logic, especially when the **SAME** value is needed in more than one place. Store such constants/config in exactly **ONE** location in the codebase, so that they stay consistent and are trivial to update later.

```

// BAD -- 0.20 is scattered across multiple files; nobody can tell WHAT it means just by
// reading it
class OrderService {
    double calculateTax(double price) { return price * 0.20; }    // magic number
}

```

```

}
class InvoiceService {
    double calculateTax(double price) { return price * 0.20; }    // same magic number,
different file
}
class ReceiptService {
    double calculateTax(double price) { return price * 0.20; }    // and again
}

// GOOD -- one named constant, in one place
public class TaxConfig {
    // Declared at class level -- accessible anywhere as TaxConfig.VAT_RATE
    public static final double VAT_RATE = 0.20;
}
// Every service now references TaxConfig.VAT_RATE instead of the literal 0.20

```

If the VAT rate ever changes (e.g., due to a change in tax law), only ONE line needs to be updated, and every part of the application that references `TaxConfig.VAT_RATE` picks up the new value automatically and consistently.

5.10 Principle 9 — Documentation

Code tells you HOW something is done; documentation tells you WHY it is done that way. Even the original author typically forgets the reasoning behind their own code within about three months, so writing down WHY decisions were made (not just WHAT the code does, which should ideally be clear from the code itself) is a genuine investment that pays off for the author too, not only for teammates.

Good documentation also lets new team members onboard significantly faster, since they can understand context and rationale without having to interview the original author (who may no longer even be on the team). The best time to document something is WHEN you write the code — writing it down later, after the context has faded from memory, is much harder and less accurate.

Things worth documenting: the details of a feature; WHY a particular business-logic decision or algorithm was adopted (not just what it does); WHY a particular external API or third-party library was chosen over alternatives; known constraints or limitations; testing status/coverage; known corner cases that are handled (or deliberately NOT handled); and any scope for future improvement that was consciously deferred.

5.11 Principle 10 — Refactoring

Continuously improving the internal structure of existing code, WITHOUT changing its observable external behavior, is itself considered a clean-coding best practice — not a separate, optional activity. This is covered in complete detail as its own topic, in Section 7.

Practice Questions — Clean Coding

A. True/False:

Statement	T / F
1. The best comment is a good name for a method or class.	
2. Comments should always be added, even if the method or variable name is already self-explanatory.	
3. SRP states that a class or function should have only one reason to change.	
4. DRY means every piece of knowledge should have a single, unambiguous representation in the codebase.	
5. Hard-coding the same magic number in several classes is an example of following the Single Source of Truth principle.	
6. A comment explaining WHY a numeric cap exists (e.g., an external API's limit) is generally considered a welcome use of comments.	

B. Define the following terminologies in brief:

Term	Definition (write yours)
SRP	
DRY	
Magic Number	
Single Source of Truth	
Self-explanatory naming	

C. Refactor the following code applying clean coding practices — meaningful naming, SRP, DRY, etc.:

```
public class OrderSystem {
    public double f1(double p, int q) {
        if (q >= 10) {
            return p * q * 0.9;
        } else {
            return p * q;
        }
    }

    public void f2(double p, int q, boolean m) {
        if (q >= 10) { // bulk order
            if (m) { // member
                System.out.println("Total: " + (p*q*0.9*0.95));
                sendReceipt();
            } else {
                System.out.println("Total: " + (p*q*0.9));
            }
        } else {
            System.out.println("Total: " + (p*q));
        }
    }
}
```

6. Code Refactoring

Refactoring is the process of changing a software system such that the EXTERNAL, observable behavior of the system does NOT change (functional requirements are preserved exactly), while the INTERNAL structure of the code is improved. Fowler describes this as "improving the design after it has been written." Refactoring formalizes and systematizes what good programmers were already informally doing — cleaning up code as they go.

6.1 Refactoring vs. performance optimization — a key distinction

Both refactoring and performance optimization share the property that functionality does NOT change. But they pull in OPPOSITE directions when it comes to readability:

Aspect	Refactoring	Performance Optimization
Goal	Make the code EASIER to understand and modify.	Make the code run FASTER (or use less memory/resources).
Effect on readability	Usually IMPROVES readability, and often reduces code size.	Often makes code HARDER to understand and read, in exchange for speed (e.g., manually unrolled loops, low-level bit tricks, caching that adds extra state to track).
When to do it	Continuously, as part of normal development (see 'when to refactor' below).	Only after profiling has shown an ACTUAL, measured performance problem in a specific hot path — optimizing blindly, without evidence, is a well-known anti-pattern ("premature optimization").

6.2 Fowler's definition and the refactoring "catalog"

Refactoring (noun): a change made to the internal structure of software to make it easier to understand and cheaper to modify, WITHOUT changing its observable behavior. Refactoring (verb): to restructure software by applying a SERIES of such refactorings, again without changing observable behavior.

Martin Fowler's book "Refactoring: Improving the Design of Existing Code" (with Kent Beck, John Brant, William Opdyke, and Don Roberts) plays a role for refactoring analogous to the role the "Gang of Four" book plays for design patterns: it provides a large, named CATALOG of specific "refactoring patterns" that programmers can refer to by name (much like design patterns), so that a team can say "let's Extract Method here" and everyone immediately understands the intended transformation.

6.3 Why refactor?

- Refactoring improves the design of software — WITHOUT active refactoring, a design naturally "decays" over time as more and more people make small, expedient changes that don't fit cleanly into the original structure.
- Refactoring makes software easier to understand, because structure improves and duplicated code is eliminated.
- Refactoring helps you FIND bugs: the deep understanding of the code that refactoring forces you to build also helps you notice existing bugs and anticipate potential future bugs.

- Refactoring helps you program FASTER, in the long run, because a good, clean design enables faster future progress (the opposite of the common but mistaken belief that "cleaning up code" is time stolen from "real" feature work).

6.4 When should you refactor?

- The Rule of Three: "three strikes and you refactor." The first time you do something, just do it. The second time you do something SIMILAR, you might wince at the duplication but do it anyway. The THIRD time you do something similar, refactor.
- Refactor when you ADD new functionality — either BEFORE adding it (to make the new function easier to slot in cleanly), or AFTER adding it (to clean up the code once the new function works).
- Refactor when you need to FIX a bug — understanding the surrounding code well enough to fix a bug correctly often naturally surfaces refactoring opportunities.
- Refactor as part of a CODE REVIEW — a second pair of eyes often spots structural issues the original author was too close to the code to notice.

6.5 The risks and practical limits of refactoring

Refactoring is not free of risk: although it helps reduce bugs OVERALL, the act of restructuring code can ITSELF introduce new bugs if done carelessly. From a manager's point of view, time spent "cleaning up the code" can look like time NOT spent implementing required features, which can create organizational friction, especially under schedule pressure.

To address these concerns, refactoring needs to be systematic, incremental, and SAFE:

- Use refactoring "patterns" (the named catalog) rather than ad-hoc restructuring, since these patterns are well-understood, well-tested transformations with known preconditions.
- Test CONSTANTLY. This ties directly into the agile-development idea of writing tests BEFORE writing the code (test-driven development): after every refactoring step, re-run the full test suite; if a test fails, the refactoring broke something, and you find out IMMEDIATELY (while the change is still small and fresh in your mind) rather than discovering it much later.

Practical limits on what CAN be refactored:

- Databases — business applications are often tightly coupled to an underlying database schema. Code is comparatively easy to change; a live production database schema, with existing data, is much harder to change safely.
- Changing interfaces — some refactorings require changing a method's signature or a class's public interface. If you own ALL the calling code, this is fine — update every caller. But if the interface is "published" (used by external code you don't control, e.g., a public library API), you cannot simply change it without breaking those external users.

6.6 Where to start: "Bad Smells" in code

A "code smell" is a surface-level indication in the source code that usually (though not always) corresponds to a DEEPER structural problem. Smells are not bugs — the code still WORKS — but they are signals worth investigating. Automated tools such as Codegrip, Checkstyle, PMD, FindBugs, and SonarQube can flag many code smells automatically as part of a build/CI pipeline.

Duplicated code

The SAME (or very similar) code appears in more than one place. This is dangerous because if you fix a bug, or update a business rule, in ONE copy but forget the others, you now have inconsistent — and possibly incorrect

— behavior scattered across the codebase. Fix with: Extract Method (pull the shared logic into one reusable method) or Pull Up Method (if the duplication is between sibling subclasses, move the shared method to their common superclass).

Long Method

A method that has grown very long is harder for a human to hold in their head all at once, and harder to test in isolated pieces. The old counter-argument ("short methods hurt performance due to call overhead") is largely OBSOLETE on modern compilers/runtimes that routinely inline small methods. Fix with: Extract Method, repeatedly, to break the long method into several well-named, smaller pieces.

Large Class

A class that has grown to try to do too much reduces cohesion (the degree to which the class's members all relate to one single, well-defined purpose). Fix with: Extract Class, splitting the class's responsibilities into two or more focused classes.

Long Parameter List

```
class PaymentService {
    public void processPayment(String name, String email,
                              String cardNumber, String expiry,
                              double amount) {
        // processing logic
    }
}
```

Long parameter lists are hard to remember the correct ORDER for, easy to accidentally pass the WRONG values into (especially when several parameters share the same type, like several Strings in a row), and this same long list is often repeated across several related methods. Fix with: Introduce Parameter Object (bundle the related parameters into their own small class, e.g., a PaymentDetails object).

Divergent Change vs. Shotgun Surgery — commonly confused with each other

Aspect	Divergent Change	Shotgun Surgery
Direction of the problem	ONE class must be edited for MANY different, unrelated reasons.	ONE kind of change requires small edits scattered across MANY different classes.
Symptom phrase	"I have to change these three methods every time I get a new database." (all in the SAME class)	A single business-rule change (e.g., a minimum order amount) requires touching validation code, display code, and email code, which all live in DIFFERENT classes.
Example	An Employee class that must be edited whenever: (a) UI/report formats change, (b) tax laws change, (c) the email-notification format changes — three unrelated reasons, one class.	Changing a minimum-purchase business rule fires changes in a validation class, a UI-messaging class, AND a pricing class simultaneously.
Underlying cause	Related to poor cohesion — the class has too MANY responsibilities bundled together (fix: Extract Class, one class per responsibility/reason-to-change).	Related to poor cohesion in the OPPOSITE direction — related logic that SHOULD live together has been scattered apart (fix: Move Method / Extract Class to consolidate the scattered logic into one place).

Feature Envy

```
// This class 'envies' the data inside Customer -- it uses Customer's data much
// more than its own, which is a signal the logic belongs on Customer instead.
public class Invoice {
    public double calculateDiscount(Customer customer) {
        if (customer.getTotalSpent() > 1000 && customer.getYearsAsMember() > 2) {
            return customer.getTotalSpent() * 0.15;
        } else if (customer.getTotalSpent() > 500) {
            return customer.getTotalSpent() * 0.10;
        }
        return 0.0;
    }
}
```

Fix: Move Method — since `calculateDiscount()` is really about Customer's own data far more than Invoice's, move the method onto the Customer class itself (e.g., `customer.calculateLoyaltyDiscount()`).

Data Clumps

The same GROUP of attributes/parameters (e.g., street, city, postalCode) keeps showing up together, over and over, across multiple method signatures or fields, but has never been grouped into its own class. Fix: Introduce Parameter Object / Extract Class, to bundle the clump into one reusable type (e.g., an Address class).

Primitive Obsession

A reluctance to use small, purpose-built CLASSES instead of raw primitive data types. Example: using three PARALLEL arrays for `std_ID[]`, `std_name[]`, and `std_marks[]` instead of a single Student class with id, name, and marks fields — parallel arrays are error-prone (easy to update one array's index but forget the others) and don't carry any of their own behavior.

Switch Statements

The same TYPE-based switch/if-else statement (e.g., "switch on the kind of shape/animal/employee-type") tends to get duplicated in multiple places around a codebase, since every new operation that depends on the type needs its own copy of the same branching logic. This can typically be replaced with POLYMORPHISM — put a virtual method on the type hierarchy itself, and let object-oriented dynamic dispatch do the "selecting" instead of an explicit conditional (see "Replace Conditional with Polymorphism" below).

Lazy Class

```
class Logger {
    // Originally meant to do complex logging (timestamps, log levels, file rotation...)
    // But now just prints a simple message
    public void log(String message) {
        System.out.println(message);
    }
}
```

A class that no longer "pays its way" — perhaps it was downsized by an earlier refactoring, or it represented planned functionality that never actually materialized. Logger here is barely doing anything beyond what `System.out.println()` already does directly, so it adds extra indirection/complexity without real benefit. Fix: Inline Class (remove the class and use its simpler replacement directly) or Collapse Hierarchy.

Message Chains ("train wrecks") and Middle Man

```
class Engine { public int getHorsePower() { return 200; } }
class Car {
    private Engine engine = new Engine();
    public Engine getEngine() { return engine; }
}
class Driver {
    private Car car = new Car();
```

```

    public void showHorsePower() {
        // Train-wreck: driver reaches THROUGH Car all the way to Engine
        System.out.println("HP: " + car.getEngine().getHorsePower());
    }
}

```

A Message Chain occurs when a client asks an object for ANOTHER object, and then asks THAT object for yet another object, chaining several `.get...()` calls together. The client now depends on the ENTIRE navigation PATH — if any intermediate relationship changes (e.g., Car no longer directly holds an Engine reference), the client code breaks, even though the client only really cared about the FINAL value (horsepower). Fix: Hide Delegate — have Car itself expose a `getHorsePower()` method that internally forwards to `engine.getHorsePower()`, so Driver only ever talks to Car.

Middle Man is almost the opposite concern: if a class is delegating MORE THAN HALF of its own responsibilities to another class (i.e., it has become mostly just a thin pass-through), you might ask whether you really need that class at all, or whether client code should just talk to the other class directly. This involves real trade-offs, though — some patterns (notably Decorator) intentionally accept a degree of "middle man"-style delegation as the whole point of the design, so this smell should be applied with judgment, not mechanically.

Inappropriate Intimacy

```

// BAD -- Order exposes ALL its internals as public fields
class Order {
    public List<Item> items = new ArrayList<>();
    public double discountRate = 0.0;
    public boolean isPaid = false;
    public String customerEmail;
}
// OrderService reaches in and manipulates Order directly
class OrderService {
    void addItem(Order order, Item item) { order.items.add(item); }           // directly
accessing
    void applyDiscount(Order order) { order.discountRate = 0.10; }           // setting
internal state freely
    void completePayment(Order order) { order.isPaid = true; }               //
changing internal flag directly
}

```

Two classes that know FAR too much about each other's private implementation details — here, Order has no real encapsulation at all, and OrderService is free to mutate its internals however it wants, meaning any invariant Order should maintain (e.g., "isPaid can only become true after a valid payment has been confirmed") can be silently violated from outside. Fix: Encapsulate Field / Move Method, giving Order proper methods (like `pay()`, `applyDiscount(rate)`) that enforce its own invariants, rather than exposing raw mutable fields.

Data Class

A class that consists ONLY of fields plus getters/setters for those fields, with NO real behavior of its own — a pure "data holder." This is not automatically bad; some classes genuinely are, and should remain, simple data holders (e.g., a small DTO used purely to transfer data across a boundary). It becomes a problem specifically when the class SHOULD have behavior (logic that naturally belongs with that data) but that behavior has instead been scattered across OTHER classes that manipulate the data class's fields from outside (which is exactly the Inappropriate Intimacy / Feature Envy pattern described above).

Comments (as a smell)

As covered in the Clean Coding topic, comments are sometimes used to hide or explain BAD code, functioning like a deodorant, rather than the underlying code being made genuinely self-explanatory through better naming and structure.

Refused Bequest

```
interface Worker { void work(); }
class HumanEmployee implements Worker {
    public void work() { System.out.println("Human working..."); }
    public void applyForLeave() { System.out.println("Applying for leave..."); }
}
class Robot implements Worker {
    public void work() { System.out.println("Robot working automatically..."); }
    // Robot has NO use for applyForLeave() if it existed on a shared superclass
}
```

A subclass IGNORES or overrides most of the functionality provided by its superclass, which is a sign that the subclass may not actually pass the "IS-A" substitutability test. (Recall: if class Y is genuinely a type of class X, then "Y IS-A X" should hold sensibly — a Dog IS-A Animal, a Car IS-A Vehicle, a SavingsAccount IS-A Account. If a supposed subclass has to override most inherited behavior into no-ops or exceptions, the inheritance relationship itself is probably modeled incorrectly.)

6.7 The catalog of refactoring patterns, in detail

Extract Method

When you have a code FRAGMENT that can logically be grouped together, turn that fragment into its OWN method, and give the new method a name that explains the PURPOSE of the fragment (not merely a description of its individual steps). This is the single most commonly applied refactoring, and is the fix for Long Method, Duplicated Code, and several other smells above.

Extract Class

When one class is doing the work of TWO (or more) distinct responsibilities, and this results in awkwardness (methods that only relate to a SUBSET of the class's fields, unclear naming, poor cohesion), create a NEW class and MOVE the fields and methods responsible for the extra responsibility into it. This is the standard fix for Large Class and Divergent Change.

Extract Variable

```
class PriceCalculator {
    public double calculatePrice(double quantity, double itemPrice) {
        return quantity * itemPrice * (quantity > 100 ? 0.9 : 0.95);
    }
}
// AFTER: extract the pieces of the hard-to-read expression into named variables
class PriceCalculator {
    public double calculatePrice(double quantity, double itemPrice) {
        double basePrice = quantity * itemPrice;
        boolean isBulkOrder = quantity > 100;
        double discountRate = isBulkOrder ? 0.9 : 0.95;
        return basePrice * discountRate;
    }
}
```

You have an expression that's HARD TO UNDERSTAND all at once; placing the result of the expression, or its individual PARTS, into separate, self-explanatory-named variables makes the logic readable step by step, without changing what it computes.

Replace Temp with Query

```
class Order {
    private int quantity; private double itemPrice;
    public double calculateTotal() {
        double basePrice = quantity * itemPrice; // temp variable -- logic is tied to this
        method only
    }
}
```

```

        if (basePrice > 1000) return basePrice * 0.95;
        else return basePrice * 0.98;
    }
}
// AFTER: the temp variable becomes a reusable method (a 'query')
class Order {
    private int quantity; private double itemPrice;
    public double calculateTotal() {
        if (getBasePrice() > 1000) return getBasePrice() * 0.95;
        else return getBasePrice() * 0.98;
    }
    private double getBasePrice() { return quantity * itemPrice; }
}

```

A local TEMPORARY variable is holding the result of an EXPRESSION. Extract that expression into its own method, and replace every reference to the temp variable with a call to the new method. The new method can then be reused by OTHER methods too, whereas the original temp variable was trapped inside just the one method it was declared in.

Move Method

```

class Engine {
    private double distance, fuelUsed;
    public double getDistance() { return distance; }
    public double getFuelUsed() { return fuelUsed; }
}
class Car {
    Engine engine;
    public double calculateFuelEfficiency() {
        // Mostly using Engine data -- a sign of Feature Envy
        return engine.getDistance() / engine.getFuelUsed();
    }
}
// AFTER: the method moves to live alongside the data it actually uses
class Engine {
    private double distance, fuelUsed;
    public double getDistance() { return distance; }
    public double getFuelUsed() { return fuelUsed; }
    public double calculateFuelEfficiency() { return distance / fuelUsed; }
}
class Car {
    Engine engine;
    public double calculateFuelEfficiency() { return
engine.calculateFuelEfficiency(); } // thin delegation
}

```

Applied when a method uses more FEATURES (attributes/operations) of ANOTHER class than of the class it is currently defined on. Create a new method with a similar body in the class it uses most, and either turn the OLD method into a simple one-line delegation (as shown above) or remove it altogether if nothing still calls it directly.

Replace Conditional with Polymorphism

You have a CONDITIONAL that chooses different behavior depending on the TYPE of an object (e.g., a switch on "bird species" to decide how fast it flies). Move each "leg" (branch) of the conditional into an OVERRIDING method on the corresponding subclass, and make the ORIGINAL method abstract on the shared superclass/interface. Object-oriented dynamic dispatch then automatically picks the correct implementation at runtime — the calling code (e.g., `printSpeed(Bird[] birds)`, which loops over birds and calls `birds[i].getSpeed()`) no longer needs any conditional logic at all, because each concrete Bird subclass already knows its own speed.

Introduce Null Object

Code that repeatedly checks a value for null (e.g., after calling `findCustomer()`) tends to scatter the SAME null-check-and-handle logic across many call sites. Rather than returning a literal null from `findCustomer()` and forcing every caller to check for it, return a special "Null Customer" object instance that implements the SAME Customer interface but with safe, neutral, do-nothing (or default) behavior. Callers can then simply call methods on the result as if it were a normal Customer — the conditional check for null disappears ENTIRELY from client code, since there's no longer any null value to check for.

Separate Query for Modifier

```
class Invoice {
    private boolean readyForSummaries = false;
    private double totalOutstanding;
    // BAD: a 'query' method that ALSO changes state -- a hidden 'side effect'
    public double getTotalOutstandingAndSetReadyForSummaries() {
        totalOutstanding = 500.0;
        readyForSummaries = true;    // side effect -- not the caller's primary intent
        return totalOutstanding;
    }
}
// AFTER: split into a PURE query (no side effects) and an explicit modifier
class Invoice {
    private boolean readyForSummaries = false;
    private double totalOutstanding;
    public double getTotalOutstanding() {           // pure query -- safe to call any time,
any number of times
        totalOutstanding = 500.0;
        return totalOutstanding;
    }
    public void setReadyForSummaries() { readyForSummaries = true; } // explicit,
intentional modifier
}
```

It is generally accepted good practice that QUERY methods (ones whose main purpose is to return information) should not have side effects — so this refactoring says to split any method that does BOTH into two clearly separate methods: one pure query, and one explicit modifier/command. Try as much as possible to avoid side effects in query methods, since a method whose name promises just "give me a value" but SECRETLY also changes state is a common, hard-to-spot source of bugs (calling it "just to check something" unexpectedly mutates the object).

Introduce Parameter Object

```
class InvoiceService {
    public double amountInvoicedIn(Date start, Date end) { /* logic using start, end */
return 1000.0; }
    public double amountOverdueIn(Date start, Date end) { /* logic using start, end */
return 200.0; }
}
// AFTER: the recurring pair of parameters becomes its own class
class DateRange {
    private Date start, end;
    public DateRange(Date start, Date end) { this.start = start; this.end = end; }
    public Date getStart() { return start; }
    public Date getEnd() { return end; }
    // behavior can grow here later, e.g.:
    public boolean includes(Date date) { return !date.before(start) && !date.after(end); }
}
class InvoiceService {
    public double amountInvoicedIn(DateRange range) { /* logic using range.getStart(),
range.getEnd() */ return 1000.0; }
    public double amountOverdueIn(DateRange range) { /* ... */ return 200.0; }
}
```

You have a GROUP of parameters that naturally go together (start and end always appear side by side, describing one conceptual date range) — this is exactly the Data Clumps smell. Bundle them into a single class instead. Note this also fixes a mild case of Primitive Obsession at the same time (using two loose Date primitives instead of a purpose-built DateRange type). The new class starts life as a simple data holder but is a natural place for related BEHAVIOR to accumulate later (see the `includes(date)` method above), which it couldn't do as two separate loose parameters.

Encapsulate Collection

If a method returns a raw, mutable COLLECTION field directly (e.g., a plain `Map` via a getter, with a matching setter that replaces the whole map), external code can freely mutate the internal collection without the owning class's knowledge or control, breaking encapsulation. Fix: make it return a READ-ONLY view of the collection, and provide explicit add/remove methods instead of a setter. Example: instead of a Student class exposing `Map getCourses()` and `void setCourses(Map courses)`, change it to expose `ReadOnlyList getCourses()`, `addCourse(Course c)`, and `removeCourse(Course c)` — now Student fully controls how its own internal collection can be modified, and can enforce any invariants (e.g., "a student cannot enroll in more than 5 courses") inside `addCourse()` itself.

Replace Nested Conditional with Guard Clauses

This refactoring is specifically about the PURPOSE of a conditional structure. There are two fundamentally different KINDS of conditional code: (1) conditionals that check for a variation in otherwise-NORMAL behavior — the system will do either A or B, and BOTH outcomes are considered equally "normal" (e.g., choosing between two valid payment methods); and (2) conditionals that check for UNUSUAL, special-case circumstances that require exceptional handling, where, if ALL of these special checks fail, the system falls through to one single, dominant "normal" path. Guard Clauses apply specifically to the SECOND kind.

```
// BEFORE -- deeply nested; hard to see what the 'normal' path even is
double getPayAmount() {
    double result;
    if (isDead) {
        result = deadAmount();
    } else {
        if (isSeparated) {
            result = separatedAmount();
        } else {
            if (isRetired) {
                result = retiredAmount();
            } else {
                result = normalPayAmount();
            }
        }
    }
    return result;
}

// AFTER -- each special case exits immediately at the top ('a guard'); the
// normal path is left completely unindented and obvious at the bottom
double getPayAmount() {
    if (isDead) return deadAmount();
    if (isSeparated) return separatedAmount();
    if (isRetired) return retiredAmount();
    return normalPayAmount();
}
```

This kind of deeply nested code sometimes originates from a novice programmer's instincts, or from a company coding standard that insists a method may only have a SINGLE return statement — a rule which, in practice, often causes MORE confusion than it prevents, since it forces exactly this kind of deep nesting purely to avoid multiple return points. After this refactoring, the method has FOUR return statements instead of one, but it is

unambiguously easier to read: every special case is handled and dismissed with one line at the top, called a "guard clause," and the true normal-case logic is left clean and unindented.

6.8 Wrapping up

Refactoring is a genuinely useful technique for making NON-functional changes to a software system that lead to better code structures, often LESS code overall, and typically triggered by the discovery of DUPLICATED code, since the refactoring catalog largely exists to show you systematic ways to eliminate duplication once you've spotted it. "Bad smells" are a useful ANALOGY, not a strict science, for discovering places in a system that are "ripe" for refactoring — the presence of a smell is a prompt to investigate, not an automatic verdict that something is broken.

Practice Questions — Refactoring

A. True/False:

Statement	T / F
1. Refactoring changes a program's external behavior in order to improve its internal structure.	
2. Duplicated code is considered a bad smell because a bug fix applied to one copy may be missed in the others.	
3. Shotgun Surgery means a single class must change for many unrelated reasons.	
4. Replace Conditional with Polymorphism is typically applied to eliminate long parameter lists.	
5. The Rule of Three suggests refactoring after the same code/structure has appeared three times.	
6. Performance optimization and refactoring both preserve functionality, but optimization often sacrifices readability while refactoring improves it.	
7. A guard clause is an early-return statement placed at the top of a method to handle a special case before the normal logic runs.	

B. Define the following terminologies in brief:

Term	Definition (write yours)
Code Smell	
Feature Envy	
Data Clumps	
Message Chain	
Extract Method	
Refused Bequest	

C. Name the code smell present, and name the refactoring pattern you would apply:

```
class ReportPrinter {  
    public void printSummary(Company company) {  
        double total = 0;  
    }  
}
```

```

        for (Department d : company.getDepartments()) {
            total += d.getBudget();
        }
        if (company.getRegion().getCountry().getCurrency().getCode().equals("USD")) {
            System.out.println("$" + total);
        } else {
            System.out.println(total + " " +
company.getRegion().getCountry().getCurrency().getCode());
        }
    }
}

```

D. Refactor this method using Guard Clauses:

```

double getPayAmount() {
    double result;
    if (isDead) {
        result = deadAmount();
    } else {
        if (isSeparated) {
            result = separatedAmount();
        } else {
            if (isRetired) {
                result = retiredAmount();
            } else {
                result = normalPayAmount();
            }
        }
    }
    return result;
}

```

7. Load Testing with Apache JMeter

7.1 What is JMeter, and why performance-test at all?

Functional testing asks "does the system produce the CORRECT output?" Performance/load testing asks a DIFFERENT question: "does the system behave ACCEPTABLY under realistic (or higher-than-realistic) levels of usage?" A feature can be 100% functionally correct for one user and still fail catastrophically under load — pages timing out, database connections exhausted, memory leaks compounding — none of which shows up in ordinary functional tests run one request at a time.

- JMeter is a dedicated performance-testing application.
- It is built using Java, so a Java SDK/runtime must be installed on the machine running JMeter.
- It is free and open source.
- It supports RECORDING real browser traffic to auto-generate a test plan, a Command Line Interface (CLI) for automation/CI pipelines, and generating HTML Reports summarizing results.

7.2 Installing and starting JMeter

- Step 1: verify Java is installed — run `java -version` in a terminal.
- Step 2: download JMeter from <https://jmeter.apache.org/>
- Step 3: unzip the downloaded archive; the JMeter folder can be kept anywhere on disk.
- Step 4: start JMeter — on Windows, run `jmeter/bin/jmeter.bat`; on Mac or Linux, run `jmeter/bin/jmeter.sh` from a terminal.

JMeter also ships with a built-in Template feature (File → Templates) that can scaffold a starting test plan for common scenarios, such as "Building a Web Test Plan," so you don't always have to assemble every element manually from scratch.

7.3 The JMeter GUI, and the overall Test-Plan hierarchy

- Menu bar — access to File, Edit, View, Help and other top-level menus.
- Toolbar — quick-access icons for common actions like New, Open, Save, and Run.
- Test Plan panel — the main working area where the test plan tree is built and configured, containing: a Tree View (a hierarchical outline of every element in your test plan, showing its structure) and a Results Tree (which displays the results once the test has actually been executed).
- Editor — selecting any single element in the tree opens its own configuration panel here, where you set that element's specific properties.

The overall MENTAL MODEL for a JMeter test plan is a strict containment hierarchy: a Test Plan contains one or more Thread Groups (each representing a pool of simulated users); each Thread Group contains one or more Samplers (the actual requests being sent) and, optionally, Config Elements, Assertions, and Non-Test Elements; and Listeners can be attached at the Thread-Group level (to capture everything that group does) or directly under a single Sampler (to capture just that one request's results).

7.4 Building a Test Plan (GUI workflow), step by step

- Step 1: Start JMeter.
- Step 2: Create a Test Plan — the root container. A test plan describes the FULL series of steps JMeter will execute when the test is run: what to test (which website/application/API), how many users to simulate, and what actions each simulated user should perform.

- Step 3: Create a Thread Group (right-click the Test Plan → Add → Threads (Users) → Thread Group), which simulates the actual USERS/load.
- Step 4: Add a Sampler under the Thread Group (right-click → Add → Sampler → e.g., HTTP Request) — this is the actual unit of work each simulated user performs.
- Step 5: Add Listeners (right-click the Thread Group or Sampler → Add → Listener) — commonly "View Results in Tree" (for detailed, per-request debugging) and "View Results in Table" (for a compact, tabular summary).
- Step 6: Run the test (via the toolbar's green "start" button) and inspect the results in whichever Listeners you added.

7.5 Thread Group — simulating users, in detail

A Thread Group simulates multiple VIRTUAL users accessing your website or application during the test. It is created via: right-click the Test Plan → Add → Threads (Users) → Thread Group, and then its own panel is used to configure the exact number of simulated users, how quickly they should all "arrive," and how many times each one should repeat its actions.

Ramp-up period — a frequently misunderstood and heavily-tested concept

The Ramp-up Period defines HOW MUCH TIME it takes JMeter to start ALL of the configured virtual users, and this startup is spread UNIFORMLY (evenly) across that time window — it is NOT an instruction about how many threads start per second directly; rather, the number of threads STARTING per second is DERIVED by dividing the total user count by the ramp-up period.

Worked calculation — a classic exam trap

If a test plan is configured for 10 users with a Ramp-up Period of 20 seconds, then on average a NEW user thread starts roughly every $20 \div 10 = 2$ seconds (equivalently, $10 \div 20 = 0.5$ new threads per second). A very common WRONG answer on this exact scenario is to claim '2 threads will be started every second' — that is backwards; it would actually take a much SHORTER ramp-up period (5 seconds, for 10 users) to reach a rate of 2 threads/second. Always compute: threads-per-second = total users $\div$ ramp-up seconds, and seconds-per-thread = ramp-up seconds $\div$ total users.

A ramp-up period exists specifically to avoid an unrealistic "instant tidal wave" of simultaneous requests hitting the server all at once (unless that sudden-spike scenario is itself exactly what you're deliberately trying to test) — real user traffic typically arrives gradually, not all in the same millisecond, so a well-chosen ramp-up produces more realistic load patterns.

7.6 Samplers — the actual requests

Samplers are the elements that actually SEND requests to the server under test and receive the response — they are the true "load-generating" unit of a test plan. They are added via: right-click the Thread Group → Add → Sampler.

- Step 1: right-click on the Thread Group, go to Add → Sampler.
- Step 2: add an HTTP Request Sampler (the overwhelmingly most common type, for testing web applications/APIs).
- Step 3: configure the sampler with the target server, path, and any parameters needed to send a real request (e.g., to Google).
- Step 4: explore the other available sampler types for non-HTTP protocols.

Other sampler types available in JMeter: Debug Sampler (useful for inspecting JMeter's own internal variables during a test run, without hitting any real server), BeanShell Sampler (runs arbitrary scripting logic), FTP Request,

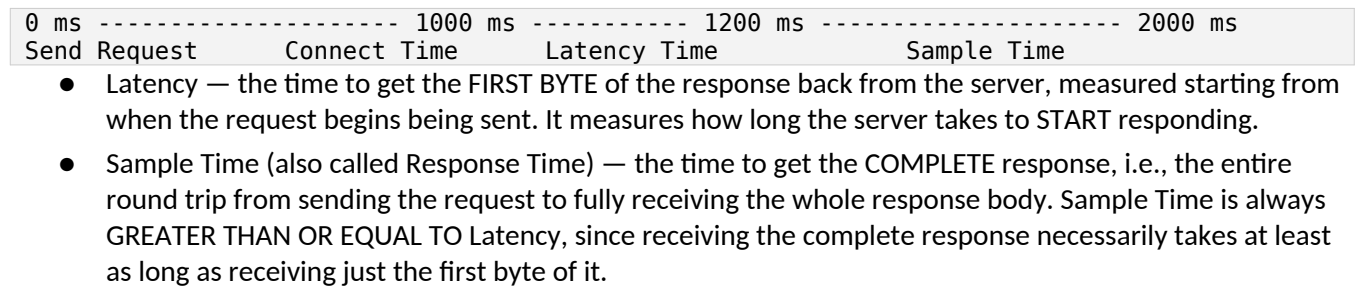
JDBC Request (for directly load-testing a database), JUnit Request (runs existing JUnit test classes as part of a load test), Java Request, LDAP Request, and Mail Reader Sampler.

7.7 Listeners — capturing and viewing results

Listeners are used for REPORTING — they capture and let you view the results of a test's execution. They are added via: right-click the Thread Group or a specific Sampler → Add → Listener → e.g., View Results Tree; you can add MULTIPLE listeners to the same run, and then run the test and inspect each one's output.

Latency vs. Sample Time — a key, precisely-defined distinction

These two metrics measure DIFFERENT spans of time during one request/response cycle, and confusing them is a very common mistake:



Some commonly used Listeners: View Results in Table (a compact spreadsheet-style row per request), View Results Tree (detailed, per-request request/response inspection — very useful for DEBUGGING why a specific request failed, but should be REMOVED or disabled for actual large-scale load runs, since it consumes significant memory), Aggregate Report (statistical summary — averages, percentiles, error rates — across all requests of a given type), Graph Results (a live visual chart of response times as the test runs), Summary Report (a lighter-weight numerical summary), and Simple Data Writer (writes raw results straight to a file, typically used together with the CLI so results can be analyzed/reported on afterward).

7.8 Assertions — validating correctness, not just speed

Assertions are checks performed on the request/response CONTENT (not on timing) to validate that the response returned is actually CORRECT, not merely that it arrived quickly. A server could respond instantly with a completely wrong or broken page — assertions are what would catch that, whereas a Listener alone would only show you a fast (but wrong) response.

- Response Assertion — checks that specific text/patterns appear (or don't appear) in the response body, headers, or response code.
- Duration Assertion — fails the sample if the response takes LONGER than a specified maximum duration.
- Size Assertion — checks that the response size (in bytes) falls within an expected range.
- HTML Assertion — validates that the response is well-formed/valid HTML.
- JSON Assertion — validates the structure/content of a JSON response, often using a JSON-path expression.
- XPath Assertion — validates XML responses using an XPath expression.

7.9 Test Script Recording — auto-generating a test plan

Rather than manually configuring every Sampler by hand, JMeter can RECORD real browser traffic and automatically build the corresponding samplers for you. The Recorder works as a local PROXY SERVER that intercepts and captures traffic between your browser and the real target website.

- Step 1: add a Recording Controller (right-click Test Plan → Add → Logic Controller → Recording Controller) — every sampler captured during recording will be placed inside this controller (you can add MULTIPLE Recording Controllers, and rename each, to organize recordings by feature/flow).
- Step 2: add a Test Script Recorder as a Non-Test Element (Test Plan → Add → Non-Test Elements → HTTP(s) Test Script Recorder) — this is the actual proxy-server component.
- Step 3: configure your BROWSER to route its traffic through JMeter as a proxy — e.g., in Chrome: click the three-dot menu → Settings → search "Proxy" → "Open your computer's proxy settings" → set a MANUAL proxy pointing at localhost on the chosen port (e.g., 8888).
- Step 4: install JMeter's own security certificate into the browser so it can also intercept HTTPS (encrypted) traffic, not just plain HTTP — in Chrome: three dots → Settings → search "certificate" → Security → Manage certificates → add the JMeter certificate under Trusted Publishers. Click Start in JMeter's recorder to GENERATE that certificate, then Stop, before installing it into the browser.

Configuring the Recorder

- Port — choose which local port the proxy server listens on (commonly 8888).
- Target Controller — under HTTP(s) Test Script Recorder → Test Plan Creation, choose which controller newly recorded requests should be placed into (typically your Recording Controller).
- Requests Filtering — exclude unwanted, noisy requests you don't actually want to load-test, such as static image files or CSS stylesheets, so the recorded test plan focuses only on the meaningful application requests.

JMeter supports two broad styles of recording: Manual (you click Start, interact with the site by hand in your browser, then click Stop) and Template-based (using a pre-built recording template for a common workflow).

Practical recording notes

- Chrome is generally considered a more security-hardened browser than Firefox, so if you hit certificate/proxy problems in Chrome, trying Firefox instead can sometimes be simpler.
- Once a manual proxy is configured in the browser, normal internet browsing will STOP WORKING unless the JMeter recorder is actually running — remember to install the certificate BEFORE switching on the proxy, and to turn the proxy setting back OFF (or start the recorder) when you're done recording.
- Recorded requests land inside their respective Recording Controller as the recording happens; after recording finishes, it's good practice to RENAME the auto-generated sampler names to something meaningful, add appropriate Listeners, and only then run the actual JMeter test.
- A convenient practice site for learning to record logins is the public "OrangeHRM demo" (search for it, or use <https://opensource-demo.orangehrmlive.com/web/index.php/auth/login>).

7.10 Command Line Interface (CLI)

The GUI mode is convenient for BUILDING and debugging a test plan, but it consumes significant memory and runs noticeably slower than the CLI — for that reason, the GUI should generally NOT be used to run actual large-scale load tests; it's meant for authoring/inspection only. The CLI ("non-GUI mode") is lightweight and integrates naturally with automated CI/CD pipelines, letting load tests run automatically as part of a build/deployment process.

```
jmeter -n -t "location of test plan file" -l "location of result file" -e -o "location of report folder"
```

Command-line flags, one by one:

- -n — run in No-GUI mode (this is what makes the run fast/lightweight).
- -t — specifies the Test plan file to run (a .jmx file — JMeter's native XML-based test-plan format).

- -l — Log results to the specified output file. The result file's extension is commonly .jtl (which can later be opened back inside JMeter's own Listener UI for inspection) but can also be .csv or other formats.
- -e — Generate an HTML report automatically once the test run finishes.
- -o — the Output folder where that HTML report will be written (this folder must NOT already exist, or JMeter will refuse to overwrite it, in most versions).
- -g — Generate a report dashboard ONLY, from an EXISTING results (.jtl) file, without actually re-running the test — useful for regenerating/reformatting a report after the fact.
- -h — display Help, listing all available command-line options.

Real-world example commands

```
# Scenario 1: Basic load test from the CLI
# Simulate 50 users concurrently accessing the OrangeHRM login page for 5 minutes
jmeter -n -t OrangeHRM.jmx -l loginResults.jtl -Jthreads=50 -Jduration=300

# Scenario 2: Running only ONE specific Test Fragment (e.g., just the login flow)
jmeter -n -t OrangeHRM_Full.jmx -l loginResults.jtl -T Login_Thread_Group

# Scenario 3: Distributed load test -- generate higher combined load by running
# from MULTIPLE machines simultaneously, coordinated by one master
jmeter -n -t OrangeHRM.jmx -r -R server1,server2 -l masterResults.jtl
```

In Scenario 1, `-Jthreads=50`` and `-Jduration=300`` pass JMeter PROPERTIES on the command line, which the test plan's own Thread Group can be configured to read (instead of hard-coding the user count/duration inside the .jmx file itself) — this lets the SAME test plan file be re-run with different load levels purely by changing the command-line arguments, without editing the file.

Putting the full GUI-workflow-to-CLI pipeline together, a typical end-to-end process looks like:

- Use the GUI to build the Test Plan → Thread Group → Sampler (HTTP Request, with protocol/server-name/path set appropriately, e.g., /moodle/, /home/, /notice/) → rename the HTTP Request sensibly.
- Add supporting Config Elements: HTTP Cookie Manager (to correctly handle session cookies across requests, as a real browser would) and HTTP Cache Manager (to realistically simulate browser caching behavior).
- Add a View Results Tree Listener and click Run in the GUI, purely to VERIFY the test plan is correctly configured before switching to the CLI for the real load run.
- Add a Duration Assertion and Save the test plan as a .jmx file.
- Switch to the CLI to actually GENERATE the report, either in one combined step (`jmeter -n -t ... -l ... -e -o ...``) or in two separate steps (first `jmeter -n -t ... -l ...`` to just run and log results, then `jmeter -g ... -o ...`` afterward to generate the report from that saved results file).

7.11 Dissecting a Web Address (JMeter's own example)

[https://www.google.com/search?](https://www.google.com/search?q=blazemeter+chrome+extension&sca_esv=314db17b76bb14ae&sxsrf=ANbLn45sKUgiBTDwijbMLikiWF2jB_LCg%3A1774693282708&source=hp)

`q=blazemeter+chrome+extension&sca_esv=314db17b76bb14ae&sxsrf=ANbLn45sKUgiBTDwijbMLikiWF2jB_LCg%3A1774693282708&source=hp`

- HTTP Protocol: https
- Server Name: www.google.com
- Path: /search
- Parameter Variables and Values (each ``&``-separated piece after the ``?``): `q = blazemeter+chrome+extension`, `sca_esv = 314db17b76bb14ae`, `sxsrf = ANbLn45sKUgiBTDwijbMLikiWF2jB_LCg%3A1774693282708`, `source = hp`, and further parameters such as `ei`, `ifsig`, `oq` (which can be empty), and `scient = gws-wiz`.

This exact skill — correctly identifying protocol, server name, path, and each parameter name/value pair from a raw URL — is directly needed both to CONFIGURE an HTTP Request Sampler correctly in JMeter (each of these pieces maps to a specific field in the sampler's configuration panel) and appears as its own standalone exam question type (see Git Section 4.13, which covers the identical concept).

7.12 Best Practices — Before, During, and After a load test

Before testing

- Get approvals and authorization from ALL relevant stakeholders before running a load test against any real environment — an unannounced heavy load test can look like (or actually cause) an outage.
- Make sure you genuinely UNDERSTAND the application being tested.
- Clearly define the REQUIREMENTS for the test — exactly what should be tested, and under what scenarios.
- Understand the client-machine configuration needed to generate the required load realistically.
- Use a dedicated TEST ENVIRONMENT, and make sure to distinguish it clearly from the PRODUCTION environment. Always run tests with the SAME infrastructure, network, and resource configuration each time, so that results are consistent and genuinely comparable against a baseline over time — changing the underlying hardware between test runs invalidates any before/after comparison.
- Plan how to manage TEST DATA, including how to clean it up after the test finishes (load tests can generate large volumes of throwaway data).
- Establish the expected BEHAVIOR, PERFORMANCE, or BASELINE ahead of time — always have a known baseline to compare new results against, or a "pass" result has no real meaning.
- Agree on a REPORTING FORMAT up front, so results are easy to share and compare later.

During testing

- Do NOT use the Test Environment for any other activity while a load test is running, since unrelated activity would contaminate the results.
- Make the test more realistic by adding THINK TIME (a deliberate pause between actions, simulating the time a real human spends reading a page before clicking the next thing).
- Add an appropriate RAMP-UP time, to give the system "breathing room" to scale up gradually, rather than an unrealistic instantaneous flood of traffic.
- Actively MONITOR the progress of the test as it runs, rather than only checking results at the very end.
- Monitor BOTH the client machine's AND the server's resources during the test (CPU, memory, network) — a bottleneck on the CLIENT side generating the load can easily be mistaken for a bottleneck on the server under test.
- Communicate promptly with the team if any issue is observed during the run.

After testing

- Analyze and DOCUMENT the results thoroughly.
- Create reporting formats appropriate for sharing with the team and other stakeholders (per the format agreed on beforehand).
- Collaborate with the team to interpret the findings and decide on next steps.
- Document your own LEARNING, both for yourself and for others who run similar tests in the future.

Practice Questions — Load Testing / JMeter

A. True/False:

Statement	T / F
1. If a test plan is run for 10 users with a ramp-up period of 20 seconds, then 2 threads will be started every second.	
2. HTTP Response Code 200 indicates that the connection could not be established.	
3. Latency is the time to get the first byte of the response from the server, while Sample Time is the time to get the complete response.	
4. The -n flag in the JMeter CLI runs the test in GUI mode.	
5. A Response Assertion checks the request/response content, while a Duration Assertion checks response time.	
6. The GUI mode is recommended for running large-scale load tests because it uses less memory than the CLI.	
7. Recording uses JMeter as a local proxy server that intercepts and captures browser traffic.	

B. Define the following terminologies in brief:

Term	Definition (write yours)
Listener	
Latency	
Assertion	
Thread Group	
Ramp-up Period	
Recording Controller	

C. Find the protocol, server name, path, parameter name(s) and parameter value(s) from the following web address:

<https://opensource-demo.orangehrmlive.com/web/index.php/auth/login?lang=en&redirect=1>

D. Write the JMeter CLI command to run a test plan located at C:\Tests\Login.jmx, log results to C:\Tests\Results.jtl, and generate an HTML report into C:\Tests\Report.

`jmeter -n -t C:\Tests\Login.jmx -l C:\Tests\Results.jtl -e -o C:\Tests\Report`

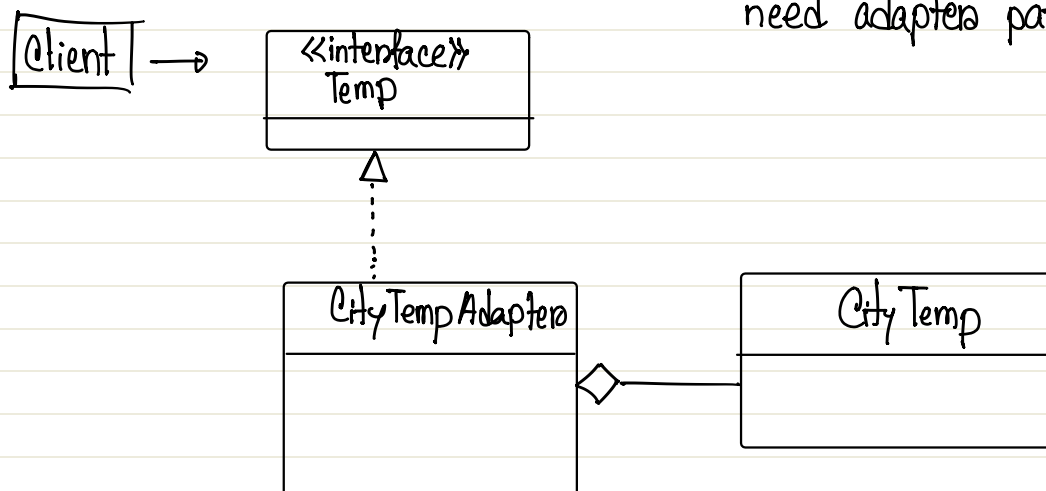
E. A test plan is run for 100 users with a ramp-up period of 50 seconds. How many users start per second on average, and roughly how long after the test starts does the 40th user start?

$$\text{users/sec} = 2 \quad \text{sec/user} = 0.5$$

$$40^{\text{th}} \text{ user} = 20 \text{ sec}$$

- (a) You have a service which returns the temperature (in celsius) by passing city name as a input value. Now, assume that your client wants to pass zipcode as input and expecting the temperature of the city in return. Develop a class diagram to present appropriate design pattern and write necessary codes so that you code fulfills all the requirements.

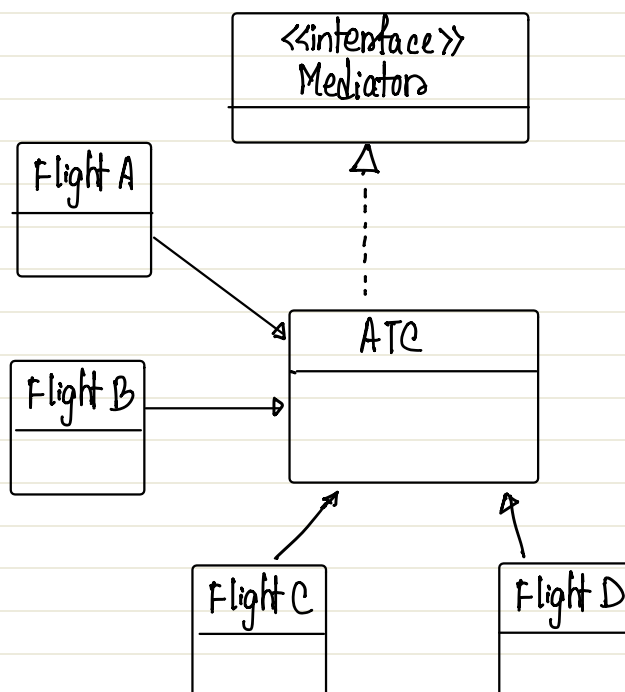
Adapters pattern.



- (a) Air traffic controller (ATC) helps in communication between flights and coordinates/controls landing, take-off. Two flights need not interact directly and there is no dependency between them. This dependency is solved by the ATC. Draw a class diagram to present appropriate design pattern and write necessary codes so that your code fulfills all the requirements.

Mediators pattern

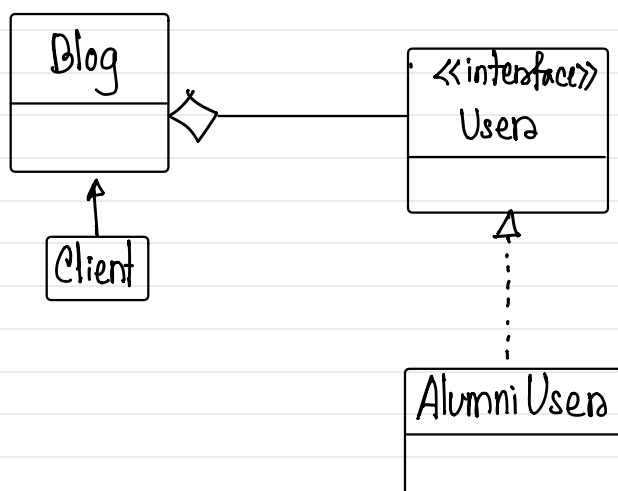
(15)



(15)

- (a) There is a blog of former BUET students. BUET Alumni register to that blog for receiving update. When a new article is posted in the blog, it will send update to the registered users saying a new article is posted. Then the user will access the blog and read the new article posted. Draw a class diagram to present appropriate design pattern and write necessary codes so that your code fulfills all the requirements following the pattern.

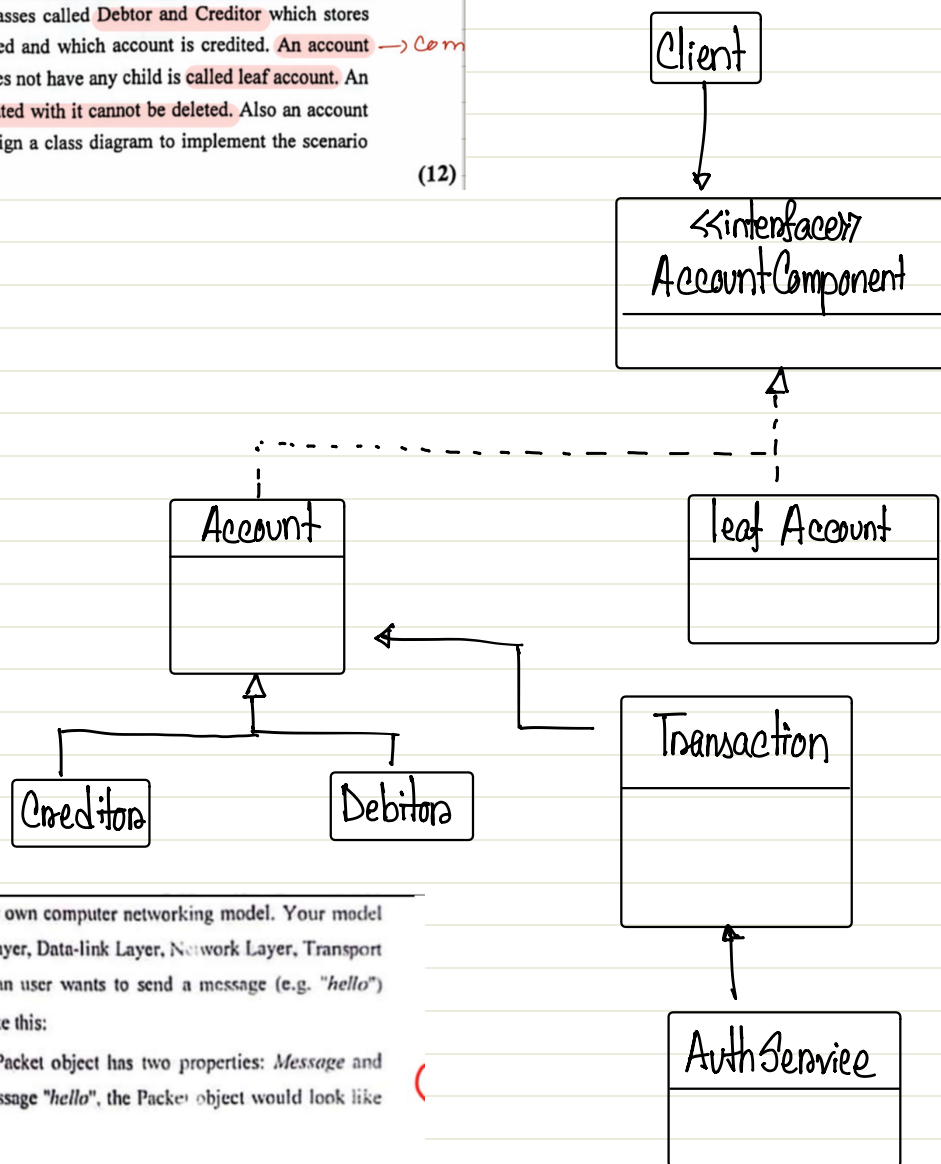
Observers Pattern



(b) Suppose we are designing an Accounting Software. We have a class named Account to represent the accounting heads in the chart of accounts. In each transaction one account is debited and at the same time another account is credited. The transaction is made from a user account authenticated using two-step authentication process (password, OTP). There are classes called Debtor and Creditor which stores the information which account is debited and which account is credited. An account may have children. The account that does not have any child is called leaf account. An account that has any transaction associated with it cannot be deleted. Also an account which has child cannot be deleted. Design a class diagram to implement the scenario described above.

(12)

Composite Pattern



(a) Let's assume you are building your own computer networking model. Your model has five (5) layers. Such as: Physical Layer, Data-link Layer, Network Layer, Transport Layer and Application Layer. When an user wants to send a message (e.g. "hello") using your designed model, it works like this:

- First user creates a *Packet* Object. Packet object has two properties: *Message* and *Length*. If user wants to send the message "hello", the Packet object would look like this:

message: hello, length: 5

- Then the *Packet* object is received by the Application layer. Application Layer adds a header to the original *Packet* object. *Packet* object would look like this:

message: APPHEADER_hello, length: 15

- Then the *Packet* object is received by the Transport layer. Transport Layer adds a header to the received *Packet* object. *Packet* object would look like this.

message: TRHEADER_APPHEADER_hello, length: 24

- Then the *Packet* object is received by the Network layer. Network Layer adds a header to the received *Packet* object. *Packet* object would look like this.

message: NETHEADER_TRHEADER_APPHEADER_hello, length: 34

- Then the *Packet* object is received by the Data-link layer. Data-link Layer adds a header and a trailer to the received *Packet* object. *Packet* object would look like this.

message: DHEADER_NETHEADER_TRHEADER_APPHEADER_hello_DTRAILER, length: 51

- Then the *Packet* object is received by the Physical layer. Physical Layer adds a header to the received *Packet* object. *Packet* object would look like this.

message: PHEADER_DHEADER_NETHEADER_TRHEADER_APPHEADER_hello_DTRAILER, length: 59

And returns this as the final *Packet* object.

Decorators

2. git checkout dev

git merge main

git checkout main

git reset --hard HEAD~3